

BrainFlow

Titolo del progetto: BrainFlow
Alunno/a: Christian Arzani
Classe: I4BB
Anno scolastico: 2025/2026
Docente responsabile: Ingrid Anna Paola Cereda

1	Introduzione	4
1.1	Informazioni sul progetto	4
1.2	Abstract	4
1.3	Scopo	4
2	Analisi	5
2.1	Analisi del dominio	5
2.2	Analisi e specifica dei requisiti	5
2.3	Use case	16
2.4	Pianificazione	17
2.5	Analisi dei mezzi	18
2.5.1	Software	18
2.5.2	Hardware	18
3	Progettazione	19
3.1	Design dell'architettura del sistema	19
3.2	Design dei dati e database	20
3.3	Design delle interfacce	21
3.3.1	Panoramica progetti	21
3.3.2	Form nuovo progetto	22
3.3.3	Board kanban del progetto	22
3.3.4	Form nuovo task	23
3.3.5	Dashboard di progetto	23
3.3.6	Gestione membri del progetto	24
3.3.7	Gestione etichette	25
3.3.8	Activity log del progetto	25
3.3.9	Vista progetto archiviato	26
3.3.10	Gestione utenti di sistema	27
3.4	Design procedurale	28
3.4.1	Creazione dei task	28
3.4.2	Scheduler controllo scadenze	29
4	Implementazione	30
4.1	Standard e convenzioni adottati	30
4.2	Sistema di autorizzazione a doppio livello	31
4.2.1	Middleware EnsureSystemRole	31
4.2.2	Middleware EnsureProjectRole	32
4.2.3	Registrazione e applicazione sulle rotte	33
4.2.4	Regole critiche	34
4.2.5	Variante di soluzione	34
4.3	Board kanban e spostamento dei task	35
4.3.1	Creazione di un task	35
4.3.2	Spostamento di un task tra colonne	35
4.3.3	Variante di soluzione	36
4.4	Activity log	37
4.4.1	Variante di soluzione	37
4.5	Notifiche email e sistema di code	38
4.5.1	Notifiche di scadenza imminente e scheduler	38
4.5.2	Code e scelta sincrona motivata	39
4.6	Validazione dei dati e gestione degli errori	40
4.7	Interfaccia grafica	41
4.8	Pannello amministrativo	42
4.9	Utilizzo dell'AI	43
5	Test	44
5.1	Protocollo di test	44
5.2	Risultati test	58
5.3	Mancanze/limitazioni conosciute	68
6	Consuntivo	69
7	Conclusioni	70
7.1	Sviluppi futuri	70

7.2	Considerazioni personali.....	71
8	Glossario	72
9	Sitografia	76
10	Allegati	77

1 Introduzione

1.1 Informazioni sul progetto

Allievo: Christian Arzani

Docente responsabile: Ingrid Paola Anna Cereda

SAMT Sezione Informatica 4BB

Inizio: 11.05.2026

Termine: 27.05.2026

1.2 Abstract

La gestione collaborativa del lavoro nei team di piccole e medie dimensioni è oggi affidata o a strumenti generici come fogli di calcolo ed email, poco strutturati e dispersivi, oppure a piattaforme commerciali come Trello, Jira e Asana, che risultano spesso piene di funzionalità per la maggior parte inutili per gruppi ristretti, molto costose dal punto di vista economico e vincolate all'archiviazione dei dati su server di terze parti. Questo progetto nasce per rispondere all'esigenza di uno strumento di project management leggero, autonomo e immediatamente comprensibile, capace di dare a team scolastici, formativi o aziendali una visione chiara dello stato di avanzamento, una netta attribuzione delle responsabilità e una notifica tempestiva degli eventi rilevanti, senza la complessità delle soluzioni esistenti. Per affrontare il problema è stata sviluppata BrainFlow, un'applicazione web basata sulla metodologia kanban, realizzata con il framework Laravel e un database relazionale MySQL. Il sistema organizza il lavoro in progetti dotati di board personalizzabili, all'interno delle quali i task sono caratterizzati da assegnatari, scadenze, priorità ed etichette. I task, inoltre, possono venire spostati tra colonne che rappresentano gli stati del flusso. La collaborazione è supportata da un activity log che traccia ogni azione significativa e da notifiche email automatiche. La gestione dei permessi è garantita da un sistema di ruoli a doppio livello, di sistema (Admin, User) e di progetto (Project Manager, Developer, Viewer). Il prodotto realizzato consente a ciascun team di gestire i propri progetti in modo indipendente e su infrastruttura propria, offrendo per ogni progetto una dashboard riepilogativa con percentuale di completamento, distribuzione dei task per stato e per membro ed evidenza delle scadenze superate. Il prodotto è uno strumento essenziale ma completo, che riduce la dispersione delle informazioni e aumenta la trasparenza all'interno del gruppo, mantenendo il pieno controllo dei dati in mano agli utenti.

1.3 Scopo

L'obiettivo di BrainFlow è fornire ai team di lavoro e agli studenti uno strumento web per la gestione collaborativa di progetti tramite board kanban, ispirato a soluzioni commerciali come Trello e Jira ma pensato sulle esigenze di team di piccole/medie dimensioni e in contesti scolastici, formativi o aziendali. Il sistema permette di organizzare il lavoro suddividendolo in progetti, board personalizzabili, task con assegnatari, scadenze, priorità ed etichette, offrendo inoltre strumenti di collaborazione come commenti con menzioni @utente, activity log per la tracciabilità delle azioni e notifiche email per gli eventi rilevanti (assegnazione di un task, avvicinamento alla scadenza, menzione in un commento, cambio di stato). Ogni progetto dispone inoltre di una dashboard riepilogativa con statistiche sull'avanzamento e sulla distribuzione del lavoro tra i membri. L'applicazione è basata su un sistema di ruoli a doppio livello: Admin e User a livello di sistema, Project Manager, Developer e Viewer all'interno del singolo progetto, permettendo una gestione granulare dei permessi. L'applicativo sarà accessibile da browser web su desktop, consentendo ai team di organizzare il proprio lavoro, ridurre la dispersione delle informazioni e aumentare la trasparenza all'interno del gruppo.

2 Analisi

2.1 Analisi del dominio

Il prodotto BrainFlow è un'applicazione web di project management collaborativo basata sulla metodologia kanban, progettata per team di piccole e medie dimensioni che necessitano di organizzare il proprio lavoro suddividendolo in progetti, board personalizzabili e task tracciabili. Il sistema integra inoltre strumenti di collaborazione come commenti con menzioni, activity log e notifiche email automatiche, accessibili tramite un'interfaccia web.

Il contesto di utilizzo è quello di team di 3-15 persone impegnati in progetti di sviluppo software, attività scolastiche o produzione di contenuti. Attualmente il problema viene risolto tramite strumenti generici come fogli Excel ed email, oppure tramite piattaforme commerciali come Trello, Jira e Asana: queste ultime offrono funzionalità avanzate ma sono spesso troppo complicate per team piccoli, prevedono costi elevati o richiedono l'archiviazione dei dati su server di terze parti.

Gli utenti finali sono studenti, sviluppatori e collaboratori di team, con competenze di base nell'utilizzo del browser e dimestichezza con applicativi web di produttività, senza necessità di conoscenze tecniche avanzate. I bisogni principali sono la visibilità immediata dello stato del progetto, la chiara assegnazione delle responsabilità e la notifica tempestiva degli eventi rilevanti.

Il dominio del project management kanban si appoggia su convenzioni, in particolare la rappresentazione del lavoro come schede su colonne corrispondenti agli stati del flusso (tipicamente Backlog, Da fare, In corso, In revisione, Completato) e il principio secondo cui ogni membro del team può consultare lo stato complessivo. Per lo sviluppo del prodotto è necessario acquisire competenze tecniche sul framework Laravel, sulla progettazione di base di dati relazionali e sui principi di sicurezza web.

2.2 Analisi e specifica dei requisiti

Il prodotto dovrà avere i seguenti requisiti:

- Sistema di autenticazione e gestione utenti con ruoli
- Gestione progetti (CRUD) con membri e ruoli dedicati
- Board kanban con colonne personalizzabili
- Gestione task con assegnazione, etichette e sotto-attività
- Commenti con menzioni e activity log per progetto
- Notifiche email per eventi rilevanti
- Dashboard riepilogativa di progetto
- Validazione dei dati, gestione errori e interfaccia intuitiva

L'utente accede all'applicazione tramite browser autenticandosi con email e password. In base al ruolo di sistema (Admin o User) e al ruolo di progetto (Project Manager, Developer, Viewer) vengono abilitate funzionalità diverse. Un Project Manager crea progetti, invita membri e configura la board kanban; i Developer gestiscono i task, li spostano tra le colonne, aggiungono etichette, sotto-attività e commenti con menzioni; i Viewer accedono in sola lettura. Ogni azione rilevante viene registrata in un activity log e può generare una notifica email. Per ciascun progetto è disponibile una dashboard con statistiche di avanzamento, esportabile opzionalmente in PDF. L'utente Admin dispone inoltre di una sezione dedicata alla gestione degli account.

ID: REQ-001	
Nome	Sistema di autenticazione e registrazione
Priorità	1
Versione	1.0
Note	
Sotto requisiti	
001	Form di registrazione con nome, email (unique), password (min. 8 caratteri, almeno 1 maiuscola e 1 numero) e conferma password
002	Form di login con email e password
003	Funzionalità di logout
004	Gestione sessione utente tramite middleware
005	Hash delle password

ID: REQ-002	
Nome	Gestione ruoli di sistema
Priorità	1
Versione	1.0
Note	I ruoli di sistema sono distinti dai ruoli di progetto (REQ-005)
Sotto requisiti	
001	Definizione di due ruoli di sistema: Admin e User
002	L'Admin può eseguire tutte le operazioni di un User per la gestione dei progetti e gestire gli account utente (CRUD)
003	L'Admin può modificare il ruolo di sistema di altri utenti
004	Pagina di amministrazione utenti accessibile solo all'Admin
005	L'utente standard di sistema (User) può comunque essere Project Manager, Developer o Viewer all'interno di un singolo progetto

ID: REQ-003	
Nome	Gestione progetti (CRUD)
Priorità	1
Versione	1.0
Note	Lo stato "archiviato" funge da soft delete
Sotto requisiti	
001	Creazione progetto con i campi: titolo (unique), descrizione (opzionale), stato (attivo/archiviato), priorità (bassa/media/alta), data di scadenza
002	Lettura: visualizzazione lista progetti a cui l'utente partecipa e dettaglio del singolo progetto
003	Modifica ed eliminazione progetto consentita al solo Project Manager o Admin
004	Cambio di stato in "archiviato" il progetto diventa di sola lettura (soft delete)
005	Prima del cambio di stato in "archiviato" deve apparire un messaggio di conferma con avvertimento dell'irreversibilità della modifica
006	Un progetto archiviato non può più essere modificato ma resta visualizzabile

ID: REQ-004	
Nome	Gestione membri del progetto
Priorità	1
Versione	1.0
Note	Dipende da REQ-003
Sotto requisiti	
001	Il Project Manager può invitare nuovi membri al progetto selezionandoli tra gli utenti registrati
002	All'invito viene assegnato un ruolo di progetto (Project Manager, Developer, Viewer)
003	Il Project Manager può modificare il ruolo di un membro
004	Il Project Manager può rimuovere un membro dal progetto
005	Visualizzazione della lista membri di un progetto con il rispettivo ruolo

ID: REQ-005	
Nome	Gestione ruoli di progetto e autorizzazioni
Priorità	1
Versione	1.0
Note	
Sotto requisiti	
001	Ruolo Project Manager: gestisce il progetto, i membri, le colonne kanban, i task e le etichette personalizzate
002	Ruolo Developer: può creare, modificare e spostare task del progetto, aggiungere commenti, creare etichette personalizzate
003	Ruolo Viewer: accesso in sola lettura a progetto, board, task, commenti, log e dashboard
004	I controlli sui permessi devono essere applicati sia a livello di rotte (middleware) sia a livello di interfaccia (nascondendo le azioni non consentite)

ID: REQ-006	
Nome	Board kanban e gestione colonne
Priorità	1
Versione	1.0
Note	Ogni progetto ha la propria board indipendente
Sotto requisiti	
001	Ogni progetto presenta una board kanban con colonne selezionabili per tipologia
002	Colonne di default selezionabili alla creazione di una colonna: "Backlog", "Da fare", "In corso", "In revisione", "Testing", "Completato"
003	Il Project Manager può aggiungere, riordinare ed eliminare colonne
004	L'eliminazione di una colonna contenente task richiede conferma, questa azione comporterà l'eliminazione delle task attribuite alla colonna

ID: REQ-007	
Nome	Gestione task (CRUD)
Priorità	1
Versione	1.0
Note	Dipende da REQ-006
Sotto requisiti	
001	Creazione task con: titolo, descrizione (opzionale), assegnatario, data di scadenza (opzionale), priorità (bassa/media/alta), etichette
002	Se non viene selezionato un assegnatario, di default viene assegnato l'utente che ha creato il task
003	L'assegnatario deve essere un membro del progetto
004	Modifica del task con gli stessi campi della creazione
005	Eliminazione del task con conferma

ID: REQ-008	
Nome	Spostamento task tra colonne
Priorità	1
Versione	1.0
Note	Tramite interfaccia grafica (drag&drop o select)
Sotto requisiti	
001	Lo spostamento del task tra colonne deve essere possibile dall'interfaccia
002	Lo spostamento aggiorna il campo column_id del task e l'ordine all'interno della colonna

ID: REQ-009	
Nome	Gestione etichette
Priorità	1
Versione	1.0
Note	Etichette di default seedate al primo avvio
Sotto requisiti	
001	Etichette di default (non eliminabili né modificabili): DIAGRAMMA, ANALISI, BACKEND, FRONTEND, DOCUMENTAZIONE, TEST
002	L'utente può creare etichette personalizzate associate al progetto con nome e colore (hex)
003	Le etichette personalizzate possono essere modificate o eliminate dai Developer o dal Project Manager
004	A un task possono essere associate più etichette

ID: REQ-010	
Nome	Sotto-attività (checklist)
Priorità	2
Versione	1.0
Note	Dipende da REQ-007
Sotto requisiti	
001	Ogni task può contenere una checklist di sotto-attività
002	Ogni sotto-attività ha titolo (obbligatorio), descrizione (opzionale) e stato completato/non completato
003	Le sotto-attività possono essere aggiunte, modificate, eliminate e contrassegnate come completate
004	Il task mostra il conteggio "completate / totali" delle sotto-attività

ID: REQ-011	
Nome	Commenti sui task con menzioni
Priorità	2
Versione	1.0
Note	
Sotto requisiti	
001	Gli utenti membri del progetto possono aggiungere commenti nella descrizione di un task
002	I commenti possono contenere menzioni nel formato @username
003	Una menzione genera una notifica all'utente menzionato (REQ-013)

ID: REQ-012	
Nome	Activity log per progetto
Priorità	1
Versione	1.0
Note	
Sotto requisiti	
001	Vengono registrati: creazione task, cambio di stato, assegnazione/riassegnazione, aggiunta commento.
002	Ogni voce contiene: utente autore, tipo di evento, task coinvolto, timestamp, dati di contesto
003	Visualizzazione del log filtrabile per progetto
004	I log sono accessibili a tutti i membri del progetto

ID: REQ-013	
Nome	Notifiche via email
Priorità	1
Versione	1.0
Note	
Sotto requisiti	
001	Notifica email all'assegnazione di un task a un utente
002	Notifica email all'avvicinamento della scadenza di un task (es. 24h prima)
003	Notifica email alla menzione in una descrizione
004	Notifica email al cambio di stato (colonna) di un task assegnato all'utente

ID: REQ-014	
Nome	Dashboard riepilogativa del progetto
Priorità	1
Versione	1.0
Note	Una dashboard per ogni progetto
Sotto requisiti	
001	Percentuale di completamento del progetto (task completati / totali)
002	Numero di task suddivisi per colonna/stato
003	Elenco task scaduti (data scadenza < oggi e non completati)
004	Numero di task assegnati per ciascun membro

ID: REQ-015	
Nome	Export report PDF
Priorità	3
Versione	1.0
Note	
Sotto requisiti	
001	Possibilità di esportare un report PDF della dashboard di progetto
002	Il report contiene: metadati progetto, statistiche dashboard, lista task con stato
003	Il PDF è scaricabile dall'interfaccia

ID: REQ-NF-001	
Nome	Validazione dei dati inseriti dall'utente
Priorità	1
Versione	1.0
Note	Copre il criterio di valutazione 194 (Attendibilità dati inseriti)
Sotto requisiti	
001	Ogni form deve validare i dati lato server.
002	I campi obbligatori sono contrassegnati visivamente nel form (es. asterisco rosso) e nella label
003	In caso di errore di validazione il campo errato viene evidenziato e viene mostrato un messaggio specifico
004	I valori già inseriti devono essere ripopolati nel form in caso di errore, senza perdita di dati per l'utente
005	Solo i dati validi vengono persistiti in database; in caso di errore nessun dato parziale deve essere salvato
006	Regole di validazione applicate: tipo (email, integer, date), range (min/max), univocità (titolo progetto), formato (hex per colori etichette)

ID: REQ-NF-002	
Nome	Gestione degli errori
Priorità	1
Versione	1.0
Note	Copre il criterio di valutazione 164 (Gestione degli errori)
Sotto requisiti	
001	Strategia di gestione errori coerente in tutto il codice: blocchi try/catch sulle operazioni critiche (invio email, transazioni DB, query complesse)
002	Pagine di errore personalizzate per HTTP 403 (Forbidden), 404 (Not Found) e 500 (Server Error)
003	I messaggi di errore mostrati all'utente sono comprensibili e non espongono dettagli tecnici interni

ID: REQ-NF-003	
Nome	Interfaccia grafica e usabilità
Priorità	1
Versione	1.0
Note	Copre il criterio di valutazione 146 (GUI, utilizzazione)
Sotto requisiti	
001	UI realizzata con Blade + Tailwind CSS per garantire coerenza visiva e velocità di sviluppo
002	Navigazione intuitiva con menù principale persistente che mostra le sezioni accessibili in base al ruolo dell'utente
003	Feedback visivo dopo ogni azione: toast / flash message di successo o errore con colorazione semantica (verde / rosso / giallo)
004	Conferma esplicita (modal) per le azioni distruttive o irreversibili: archiviazione progetto, eliminazione task, rimozione membro, eliminazione colonna
005	Layout responsive funzionante su desktop e tablet, su mobile è accettabile uno scroll orizzontale della board
006	Elementi UI con stati visivi chiari: hover, focus, disabled, loading
007	Labels e placeholder dei campi chiari e auto-esplicativi

ID: REQ-NF-004	
Nome	Utilizzabilità in produzione del prodotto
Priorità	1
Versione	1.0
Note	Copre il criterio di valutazione 119 (Utilità applicazione)
Sotto requisiti	
001	L'applicazione al termine del LPI deve essere installabile e funzionante in un ambiente di produzione senza modifiche al codice
002	Tutti i requisiti funzionali di priorità 1 devono essere completamente implementati e funzionanti
003	È fornita una guida d'installazione (allegato) con: prerequisiti, comandi di installazione, configurazione .env, esecuzione migrazioni e seeders
004	Le migrazioni del database creano correttamente l'intero schema e i seeders popolano i dati di base (etichette di default, account Admin iniziale)
005	Il sistema gestisce correttamente i casi limite: progetto senza task, board senza colonne personalizzate, task senza scadenza, utente senza progetti assegnati

ID: REQ-NF-005	
Nome	Sicurezza dell'applicazione
Priorità	1
Versione	1.0
Note	Misure minime di sicurezza per un'applicazione web multi-utente
Sotto requisiti	
001	Tutte le route che richiedono autenticazione sono protette dal middleware auth di Laravel
002	Controlli di autorizzazione per ruolo (Admin, Project Manager, Developer, Viewer)
003	Protezione contro CSRF abilitata su tutti i form (token CSRF di Laravel)
004	Protezione contro SQL injection garantita dall'uso esclusivo di Eloquent ORM e Query Builder con parametri
005	Protezione contro XSS: output Blade tramite la sintassi {{ }} che effettua escape automatico
006	Password salvate in DB esclusivamente come hash bcrypt; non sono mai visibili in chiaro, log o response

2.4 Pianificazione

Per la pianificazione del progetto è stato scelto il modello di sviluppo Waterfall, ritenuto adeguato visto il poco tempo a disposizione per svolgere il progetto.

Il diagramma di Gantt rappresenta la suddivisione delle 80 ore di lavoro distribuite su 3 settimane (effettive sono 2 settimane causa ponti). Il progetto è suddiviso in 4 fasi:

- **Analisi e Progettazione (12h, 15%)**: raccolta dei requisiti, la stesura dei diagrammi di progettazione e pianificazione
- **Implementazione (48h, 60%)**: il capitolo d'implementazione raccoglie delle task generiche che poi si definiscono più nel dettaglio in una board Trello. La percentuale di questa fase è aumentata da 55% al 60% in accordo con il perito 1 e la docente responsabile, siccome ho ritenuto che il tempo a mia disposizione non sarebbe stato abbastanza, ovviamente questo è andato a discapito della percentuale della documentazione che è passata da 20% a 15%.
- **Test (8h, 10%)**: fase che si svolge durante una giornata che ha come scopo lo svolgimento dei test e il fix di eventuali bug
- **Documentazione (16h, 15%)**: si svolge parallelamente alle altre fasi durante tutto il progetto, con una concentrazione finale negli ultimi due giorni dedicata alla rifinitura, alla stesura dei capitoli mancanti e alla preparazione della consegna.

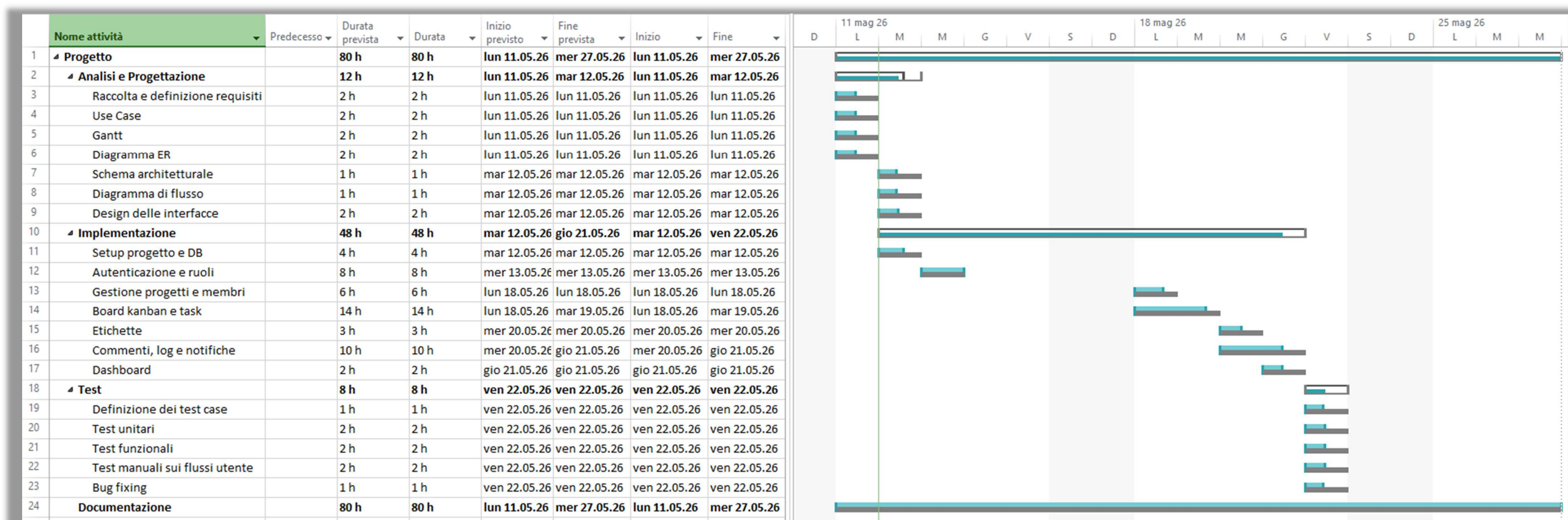


Figura 2 - Gantt preventivo

2.5 Analisi dei mezzi

La selezione dello stack tecnologico è stata fatta privilegiando soluzioni mature che coprissero nativamente i requisiti senza introdurre complessità inutile, questo per rispettare le tempistiche. Laravel è stato adottato come framework principale perché offre già integrati ORM, migrazioni, validazione, controllo degli accessi, notifiche email e scheduler, permettendo di concentrare il tempo sulla logica applicativa anziché sull'infrastruttura. Su questa base, MySQL è stata una scelta abbastanza scontata siccome è integrabile molto velocemente con Laravel. Per il mail server è stato scelto Google perché è un provider affidabile, ovviamente con il poco tempo a disposizione, il servizio di Google ha evitato la configurazione di un server di posta proprio. C'è da considerare però anche i limiti che Google pone agli utenti con il piano gratuito, le limitazioni principali sono: 500 mail giornaliere e limite di destinatari abbastanza ristretto (non viene specificato di preciso). Questi limiti ovviamente sarebbero un ostacolo in produzione, ma per un progetto piccolo come questo è più che sufficiente. Per quanto riguarda l'autenticazione è stato scelto Laravel Breeze che fornisce uno pacchetto già preconfigurato, perfetto come base su cui adattare la struttura dell'applicativo web. Per quanto riguarda lato client è stato scelto Alpine.js che serve per aggiungere il comportamento dinamico necessario (modali, drag & drop), integrandosi con Blade senza il peso di un framework aggiuntivo più pesante.

2.5.1 Software

- **PHP 8.2** — linguaggio del backend.
- **JavaScript** — per l'interattività lato client.
- **Laravel 12** (v12.58.0) — framework PHP MVC che costituisce l'applicazione: routing, ORM Eloquent, sistema di migrazioni, validazione, middleware, sistema di notifiche e scheduler.
- **Laravel Breeze** (v2.4.1) — pacchetto minimale per l'autenticazione (registrazione, login, logout, reset password, verifica email), adottato come base e poi personalizzato.
- **Blade** — template engine integrato in Laravel, usato per la generazione delle viste lato server.
- **Tailwind CSS 3** (^3.1.0) — framework CSS utility-first per lo stile dell'interfaccia, con il plugin **@tailwindcss/forms** (^0.5.2) per la formattazione coerente dei campi dei form.
- **Alpine.js 3** (^3.4.2) — micro-framework JavaScript per l'interattività leggera (modali, menu a tendina, drag & drop dei task) senza introdurre un framework SPA pesante.
- **Vite 7** (^7.0.7) — bundler e dev server per la compilazione e l'ottimizzazione di CSS e JavaScript, integrato tramite **laravel-vite-plugin** (^2.0.0).
- **Axios** (^1.11.0) — client HTTP per le richieste asincrone dal frontend verso le API interne (es. spostamento task, gestione membri).
- **MySQL 8.0** — database relazionale utilizzato nell'ambiente di produzione
- **SMTP** mail server di Google, per le notifiche di assegnazione, scadenza, spostamento e invito.
- **PHPUnit** (11.5.55) — framework per i test automatici (Unit e Feature).
- **FakerPHP/Faker** (v1.24.1) — generazione di dati fittizi per i seeder e i test.
- **PhpStorm** (2025.1.1) — scrittura del codice
- **Brave Browser** (1.90.124) — navigazione web
- **MySQL Workbench** (8.0.42) — gestione database e testing

2.5.2 Hardware

Workstation Principale:

- CPU: Intel(R) Core(TM) i7-9700 CPU @ 3.00GHz
- RAM: 32.0 GB Disco: 500GB
- GPU: NVIDIA GeForce RTX 2060

3 Progettazione

3.1 Design dell'architettura del sistema

Dal lato utente, il Client è la postazione di lavoro dalla quale si accede all'applicazione tramite un Browser Web (Chrome, Firefox, Edge o equivalenti). Le richieste HTTP/HTTPS viaggiano attraverso Internet fino al Web Server, che ospita l'applicazione Laravel e si occupa di elaborare le richieste, eseguire la logica applicativa e restituire le pagine HTML generate dal template engine Blade.

Il Web Server è collegato direttamente al Database MySQL, che si occupa della gestione dei dati.

Per l'invio delle notifiche email, il Web Server comunica con un servizio SMTP esterno, nello specifico il mail server di Google, raggiunto attraverso Internet. Questa scelta permette la gestione della consegna delle email a un provider affidabile, evitando la necessità di mantenere un mail server proprio.

Da notare la ridondanza di Internet nello schema, questo viene fatto per evitare fraintendimenti su possibili interazioni del client con il mail server.

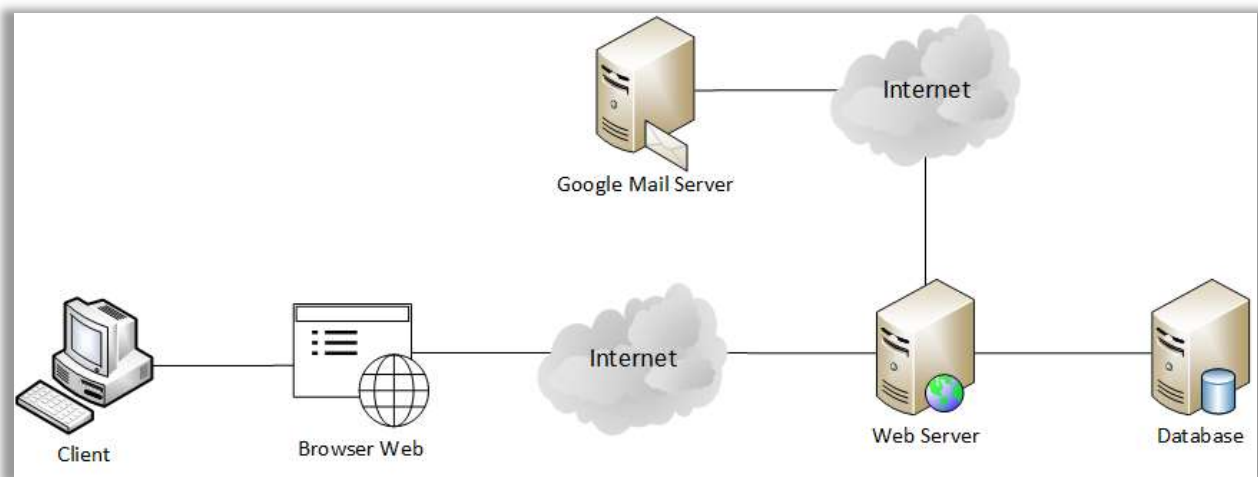


Figura 3 - Diagramma di architettura

3.3 Design delle interfacce

Il design progettato punta ad essere fedele a quello che sarà il risultato finale dell'applicativo. L'obiettivo della grafica è fornire all'utente un'interfaccia pulita, professionale e immediatamente comprensibile, con un'estetica sobria ma curata; il colore principale scelto è un indaco/viola scuro, accompagnato da una palette neutra di grigi su sfondo chiaro che mette in risalto i contenuti senza appesantire la lettura.

La struttura dell'applicazione segue un layout a due colonne: un menu laterale fisso sulla sinistra, che consente di navigare tra le sezioni principali (Progetti, Utenti, progetti attivi e archivio) ed è sempre accessibile, e un'area centrale di lavoro sulla destra, che cambia in base alla sezione selezionata. All'interno di ogni progetto è presente una navigazione secondaria a schede (Board, Dashboard, Membri, Etichette, Activity) che permette di muoversi rapidamente tra le diverse viste dello stesso progetto. Le informazioni sono organizzate in card e box ben spaziati per facilitare il riconoscimento visivo dei dati e dei raggruppamenti logici.

3.3.1 Panoramica progetti

Pagina di atterraggio dell'applicazione, mostra l'elenco dei progetti a cui l'utente partecipa sotto forma di card con priorità, scadenza, membri e percentuale di completamento.

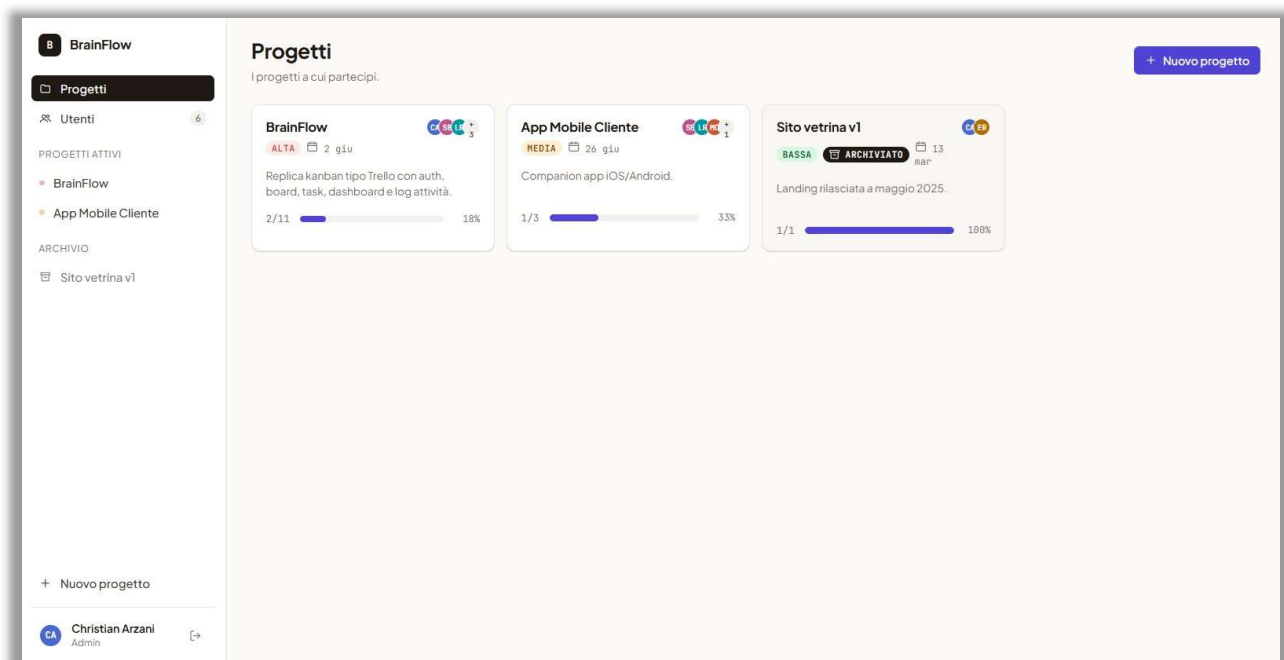
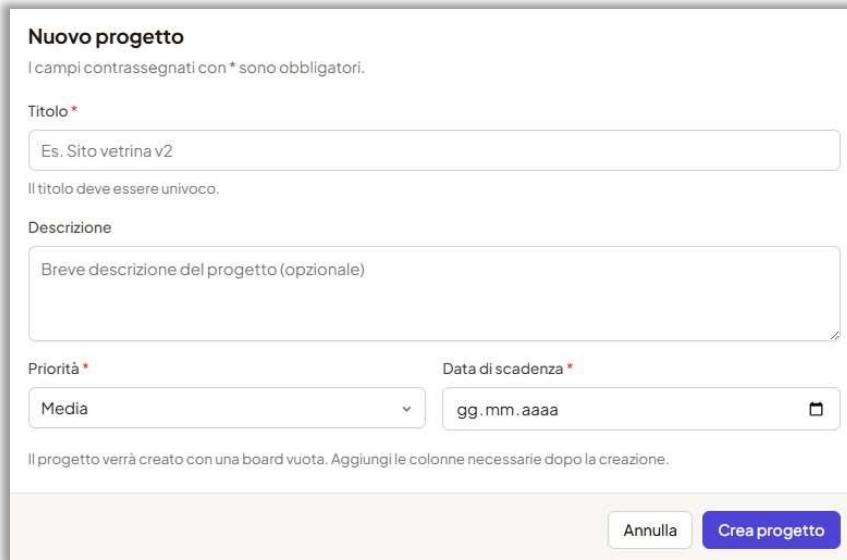


Figura 5 - Panoramica progetti

3.3.2 Form nuovo progetto

Modale per la creazione di un nuovo progetto con i campi titolo, descrizione, priorità e data di scadenza.



Nuovo progetto

I campi contrassegnati con * sono obbligatori.

Titolo *

Es. Sito vetrina v2

Il titolo deve essere univoco.

Descrizione

Breve descrizione del progetto (opzionale)

Priorità *

Media

Data di scadenza *

gg. mm. aaaa

Il progetto verrà creato con una board vuota. Aggiungi le colonne necessarie dopo la creazione.

Annulla Crea progetto

Figura 6 - Form nuovo progetto

3.3.3 Board kanban del progetto

Vista principale del progetto, mostra le colonne configurabili e i task come card spostabili tramite drag&drop tra le colonne.

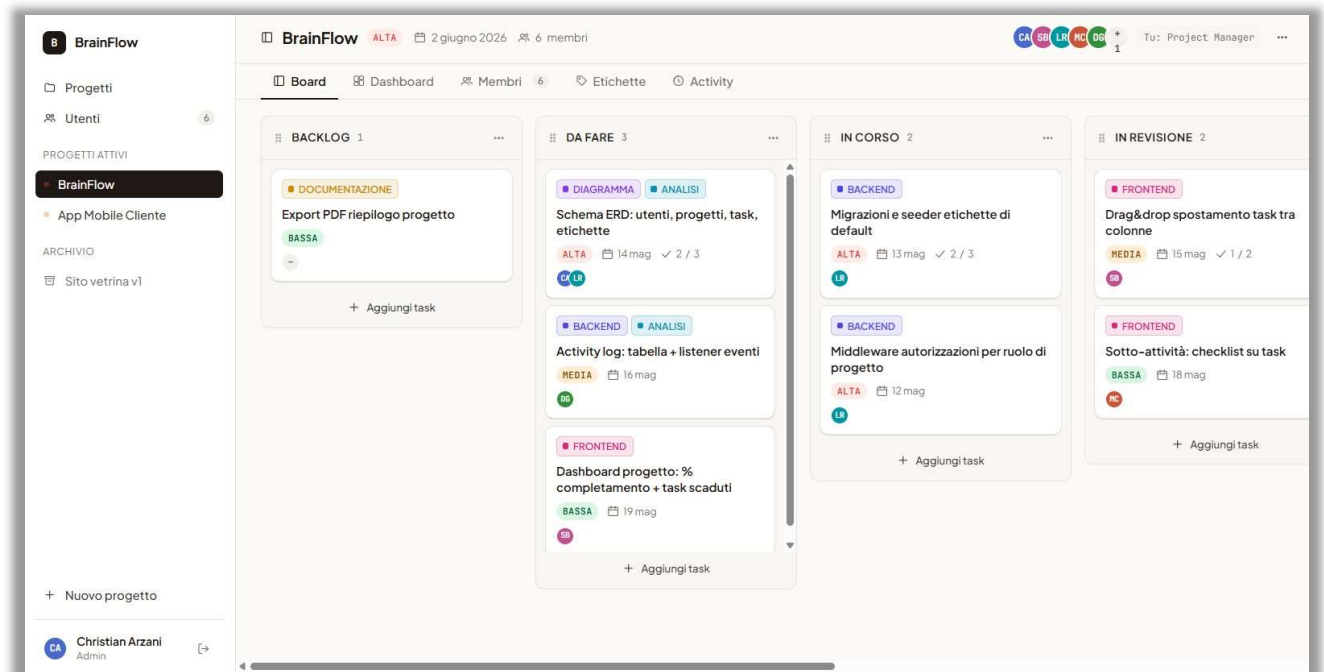


Figura 7 - Board kanban del progetto

3.3.4 Form nuovo task

Modale per la creazione di un task con titolo, descrizione, priorità, scadenza, assegnatari ed etichette, comprese le indicazioni per l'uso delle menzioni @username.

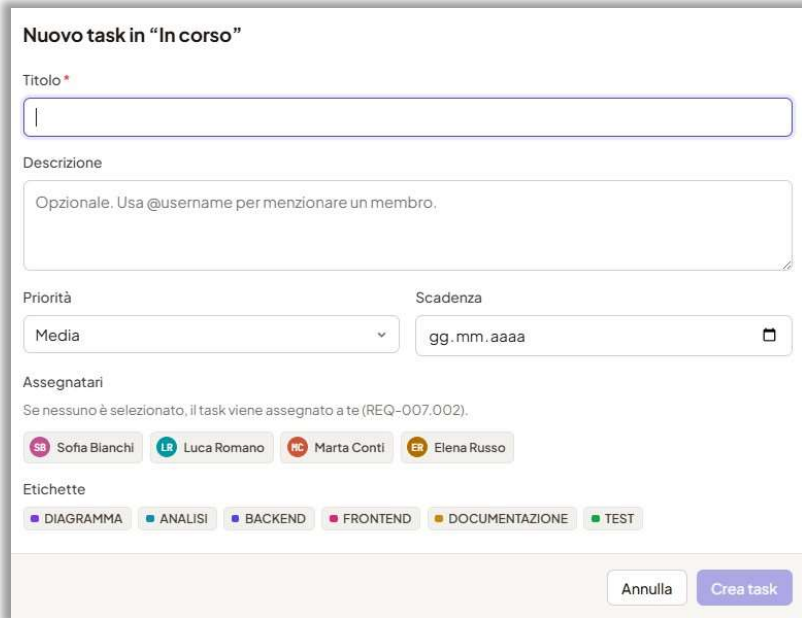


Figura 8 - Form nuovo task

3.3.5 Dashboard di progetto

Riepilogo statistico del progetto con percentuale di completamento, task totali, task scaduti, membri attivi e distribuzione dei task per colonna e per assegnatario.

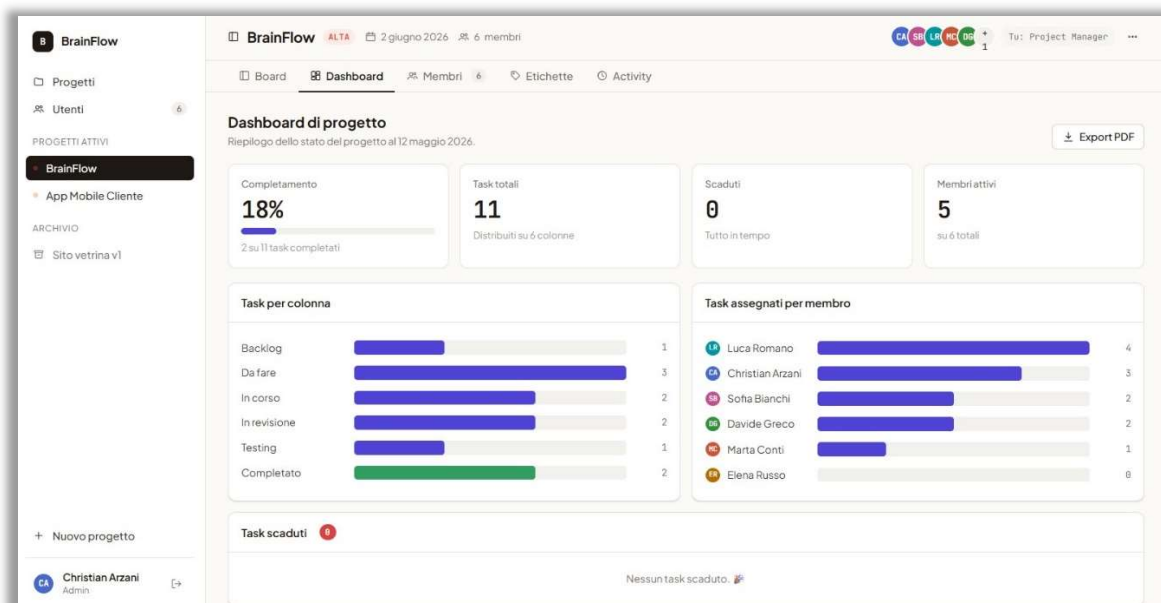


Figura 9 - Dashboard di progetto

3.3.6 Gestione membri del progetto

Elenco dei membri del progetto con il rispettivo ruolo, modificabile dal Project Manager, e pulsante per invitare nuovi membri.

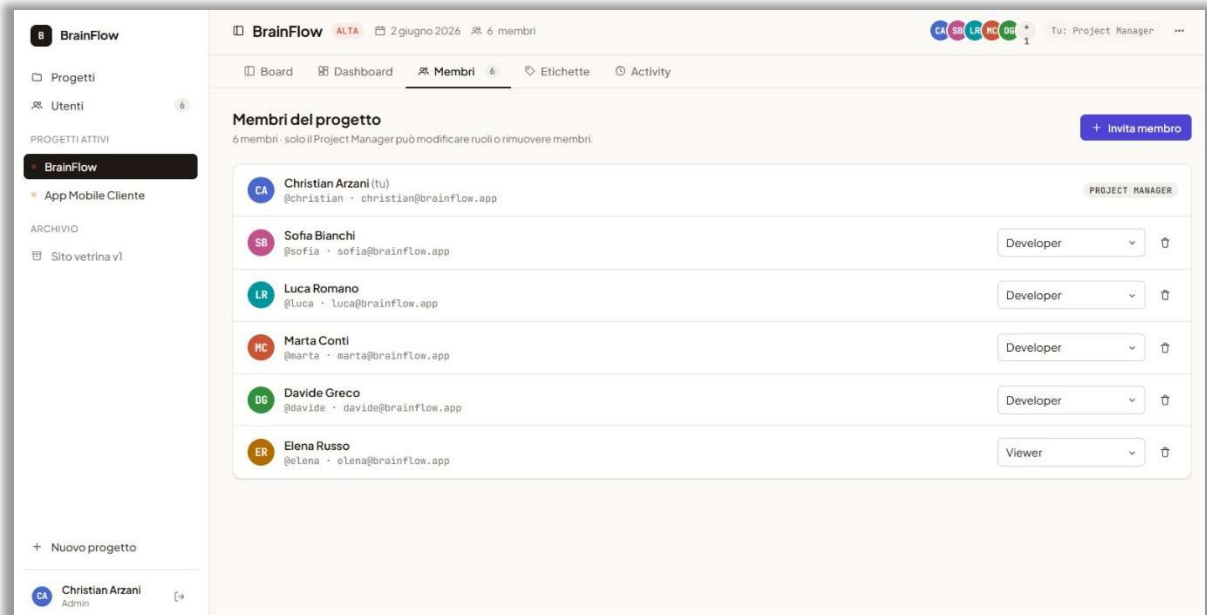


Figura 10 - Gestione membri del progetto

3.3.7 Gestione etichette

Pagina che mostra le etichette di sistema e le etichette personalizzate del progetto, con possibilità di crearne di nuove e di eliminare quelle custom.

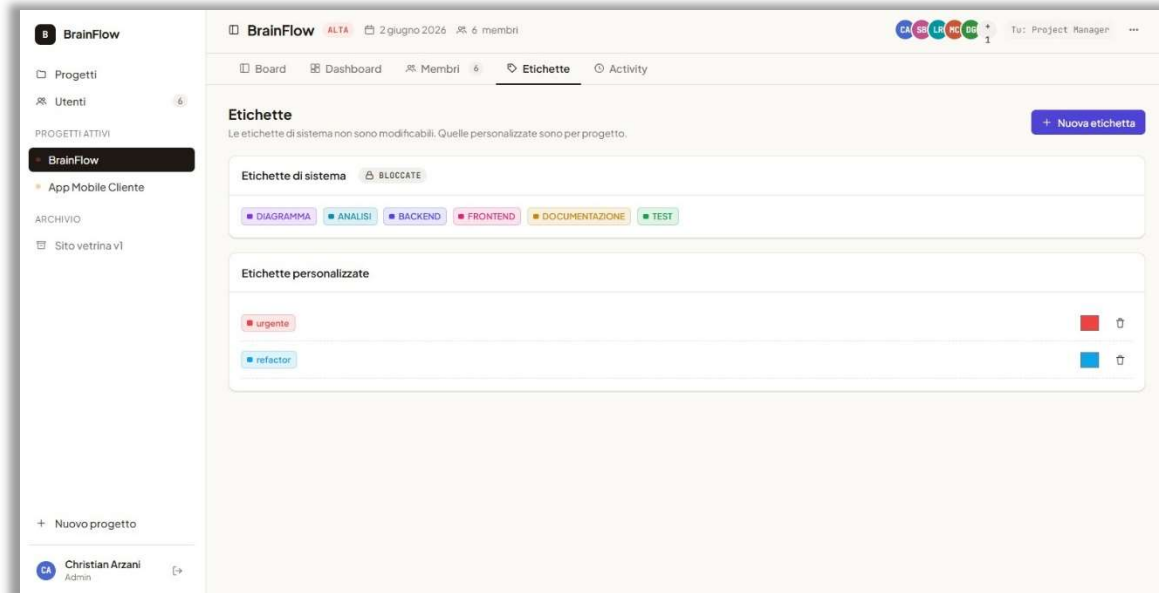


Figura 11 - Gestione etichette

3.3.8 Activity log del progetto

Cronologia di tutte le azioni rilevanti effettuate sul progetto (creazione e spostamento task, assegnazioni, commenti, ecc.) con autore, descrizione e timestamp relativo.

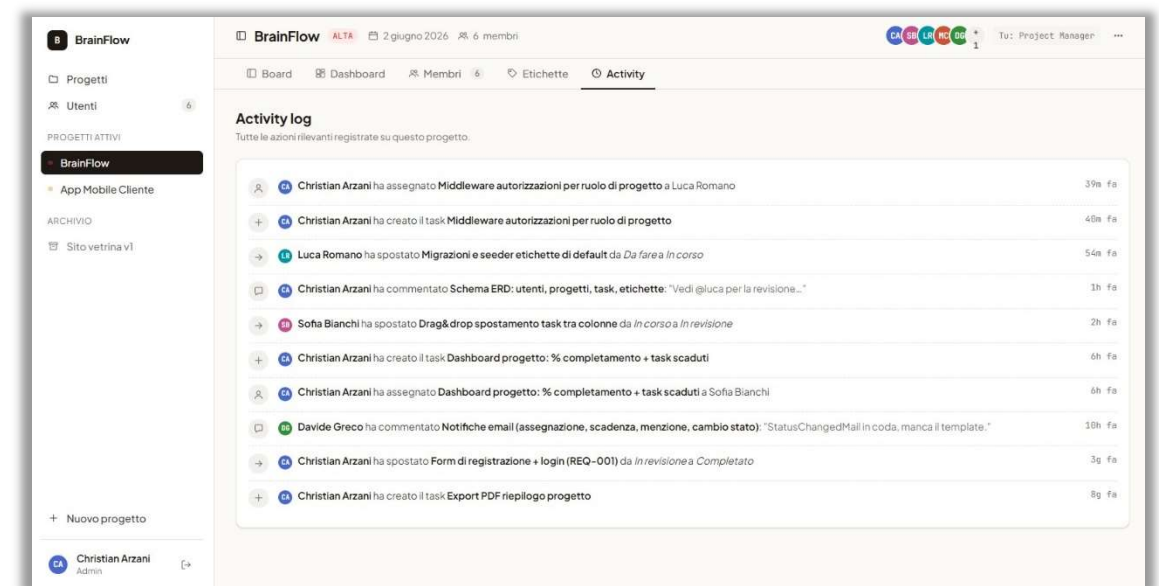


Figura 12 - Activity log del progetto

3.3.9 Vista progetto archiviato

Vista in sola lettura di un progetto archiviato, con banner di avviso che informa l'utente dello stato di soft delete e dell'impossibilità di apportare modifiche.

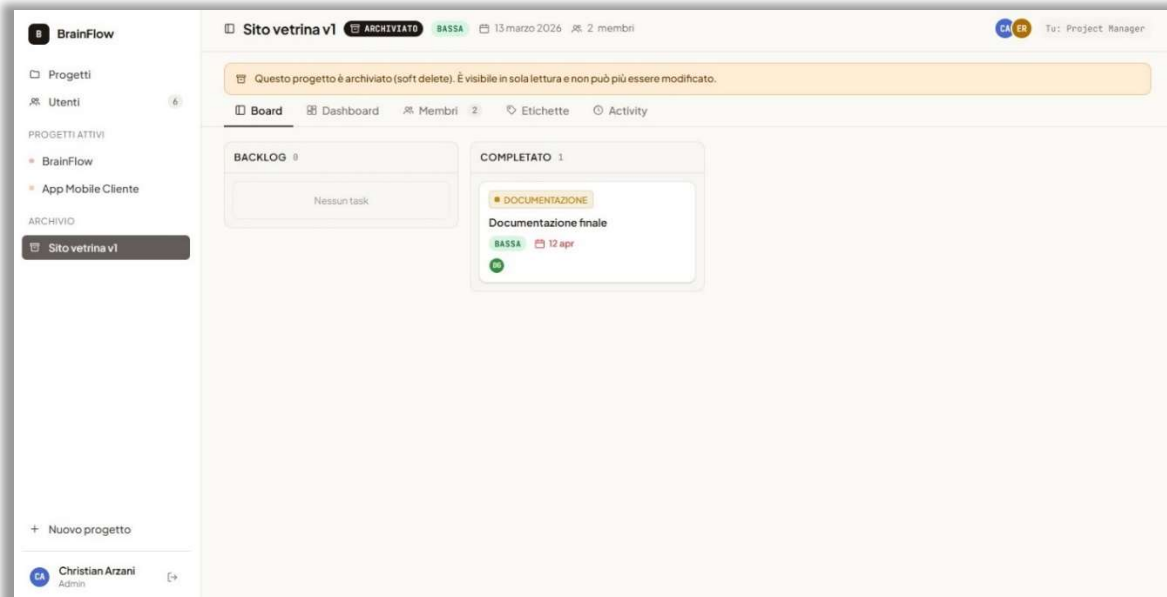


Figura 13 - Vista progetto archiviato

3.3.10 Gestione utenti di sistema

Pagina riservata all'Admin che mostra l'elenco di tutti gli account registrati e permette di modificarne il ruolo di sistema (Admin o User).

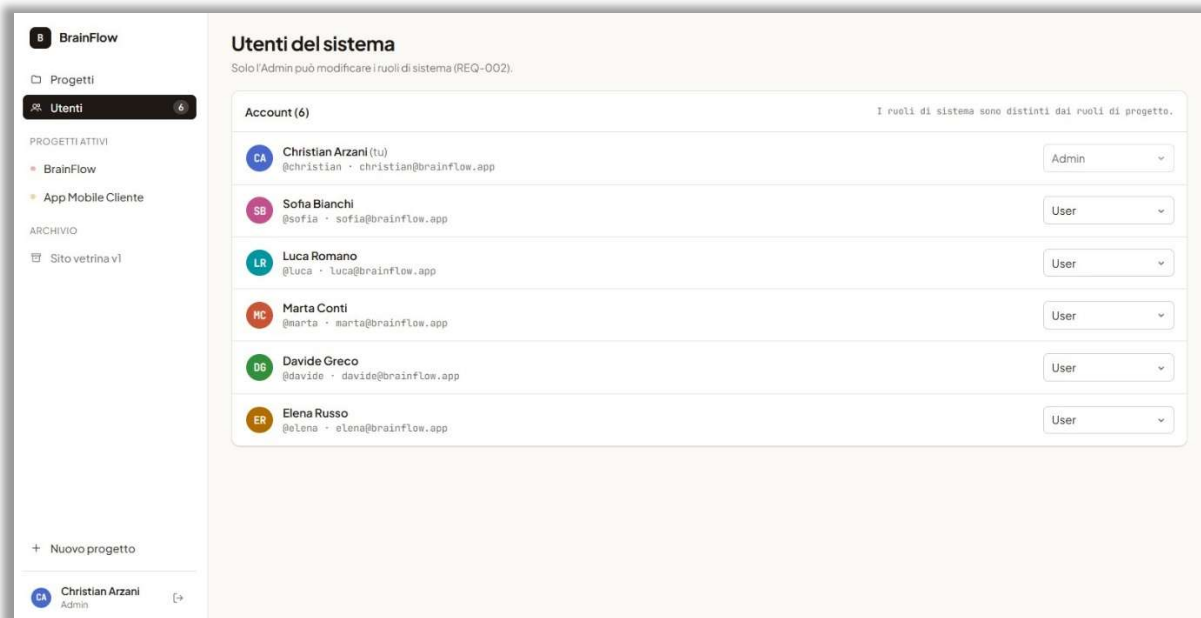


Figura 14 - Gestione utenti di sistema

3.4 Design procedurale

I diagrammi procedurali progettati per questo progetto sono dei FlowChart (diagrammi di flusso) e rappresentano i processi più significativi e non evidenti dell'applicativo: creazione di un task con sistema di notifiche e scheduler per il controllo delle date di scadenza.

3.4.1 Creazione dei task

Viene descritto il processo di creazione di un task all'interno di un progetto, dai controlli di autenticazione e autorizzazione fino al salvataggio e all'invio della notifica all'assegnatario. Il risultato del processo è un nuovo task inserito nel database, tracciato nell'activity log e notificato via email.

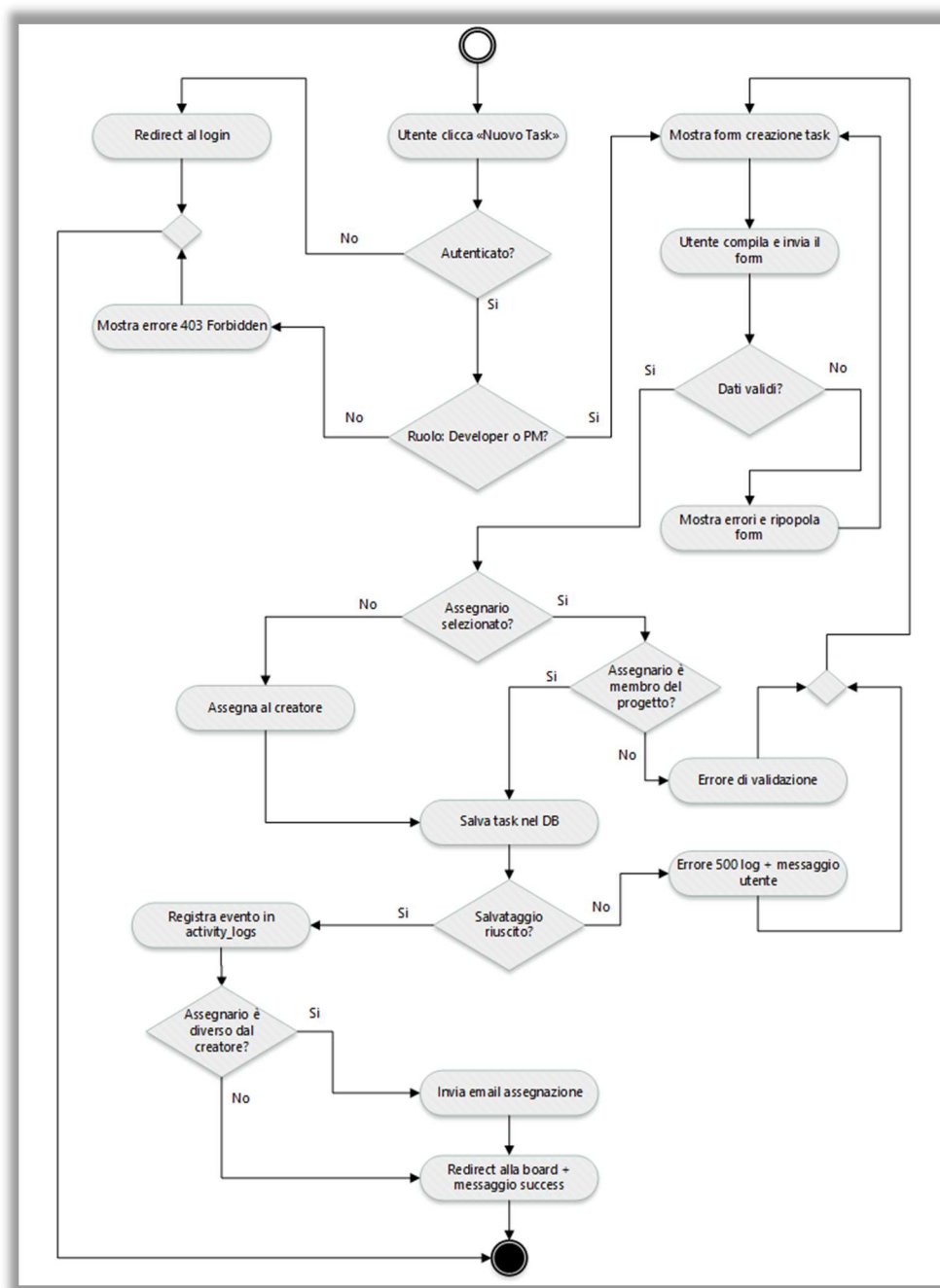


Figura 15 - FlowChart, Creazione di un task

3.4.2 Scheduler controllo scadenze

Viene descritto il processo automatico, eseguito periodicamente dallo scheduler, che individua i task in scadenza nelle 24 ore successive e invia le relative notifiche email agli assegnatari. Il risultato del processo è l'invio delle notifiche di scadenza, evitando duplicati grazie al tracciamento sulla tabella notifications.

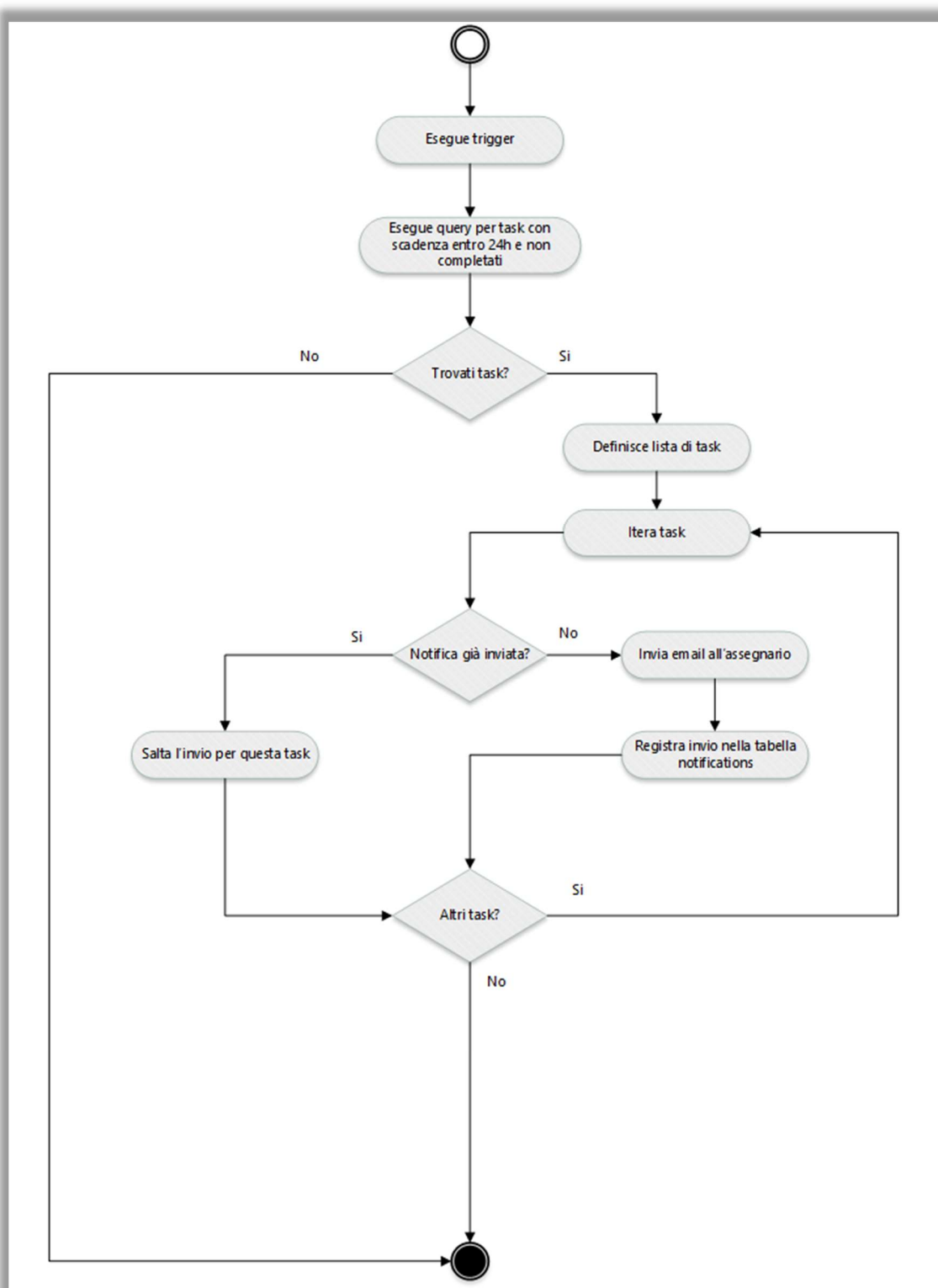


Figura 16 - FlowChart, Controllo scadenze scheduler

4 Implementazione

L'applicazione è costruita sul framework Laravel seguendo il pattern architetturale MVC. La logica è distribuita tra Controller, che gestiscono le operazioni, classi di validazione dedicate (Form Request), middleware per il controllo degli accessi e Model Eloquent che permette la gestione delle relazioni e dei dati del database.

4.1 Standard e convenzioni adottati

Lo sviluppo ha seguito un insieme di standard e convenzioni riconosciuti, sono stati scelti con l'obiettivo di garantire leggibilità, manutenibilità e coerenza del codice in un progetto che nonostante sia individuale è stato pensato per continuarlo anche dopo la conclusione. Leggendo il codice sembrerebbe che la coerenza del codice non sia stata rispettata per via dell'utilizzo di variabili e funzioni in lingua inglese che vanno in contrasto con la stesura dei commenti in lingua italiana, questo è stato fatto per aiutare i prossimi studenti che prenderanno in mano questo progetto a capire di che cosa si stia parlando avendo però una struttura universale in lingua inglese. Gli standard e le convenzioni sono le seguenti:

- **PSR-12** come standard di stile per il codice PHP (indentazione a 4 spazi, posizione delle parentesi, spaziatura): garantisce che il codice sia scritto "ovunque nello stesso modo" e quindi prevedibile per chiunque lo legga.
- **Convenzioni di naming di Laravel:** Model al singolare (Task, Project), tabelle al plurale (tasks, projects), tabelle pivot in ordine alfabetico (project_user, task_user, label_task), metodi dei resource controller standard (index, store, update, destroy). Questo permette a Laravel di risolvere automaticamente molte associazioni e riduce la quantità di configurazione esplicita.
- **Architettura MVC con Form Request e Middleware:** la validazione è estratta dai controller in classi dedicate, e il controllo dei permessi è centralizzato in middleware. I controller restano così focalizzati solo sulla gestione della logica.
- **Route RESTful:** le rotte seguono lo standard HTTP (POST per creare, PATCH per aggiornare, DELETE per eliminare) e una struttura gerarchica delle risorse (es. /projects/{project}/tasks/{task}/move).
- **PSR-4** che definisce come caricare automaticamente (autoloading) le classi PHP a partire dai loro namespace e dalla struttura delle cartelle (App\Http\Controllers, App\Models, App\Notifications, ...).

La maggior parte di queste convenzioni è imposta di default: aderirvi significa lavorare con la struttura di Laravel anziché contro, ottenendo un codice più compatto e meno soggetto a errori.

4.2 Sistema di autorizzazione a doppio livello

Il controllo degli accessi è il meccanismo più importante dell'applicazione, perché opera su due livelli indipendenti. Il ruolo di sistema (campo `sys_role` sulla tabella `users`, con valore `user` o `admin`) determina cosa l'utente può fare a livello globale, mentre il ruolo di progetto (`pm`, `developer` o `viewer`) è memorizzato nella tabella pivot `project_user` e vale solo all'interno del singolo progetto. Lo stesso utente può quindi essere Project Manager in un progetto e semplice Viewer in un altro. Questa separazione consente una gestione scalare dei permessi senza confondere i ruoli globali con quelli di progetto.

4.2.1 Middleware EnsureSystemRole

Il primo middleware protegge le rotte riservate al ruolo di sistema. Accetta una lista variabile di ruoli ammessi, la classe è stata pensata per essere utilizzata in qualsiasi rotta.

```
christian.arzani
17 public function handle(Request $request, Closure $next, string ...$roles): Response
18 {
19     $user = $request->user();
20
21     if (!$user) {
22         abort( code: 401, message: 'Non autenticato.');
```

Figura 17 - EnsureSystemRole, controllo del ruolo di sistema

4.2.2 Middleware EnsureProjectRole

Il secondo middleware risolve il ruolo dell'utente all'interno del progetto leggendolo dalla tabella pivot. Un dettaglio da notare importante è il controllo che viene fatto per l'amministratore di sistema. Un admin ha accesso a qualsiasi progetto senza dover essere presente nella pivot, evitando di doverlo iscrivere manualmente a ogni progetto.

```
// Admin di sistema vede tutto, senza essere nella pivot
if ($user->sys_role === 'admin') {
    $request->attributes->set('project_role', 'admin');
    return $next($request);
}
```

Figura 18 - EnsureProjectRole, controllo sul ruolo di sistema admin

Oltre al controllo del ruolo di sistema viene controllata l'esistenza del progetto, questo per gestire l'errore ed evitare che l'utente mandi richieste invalide senza avere un feedback. Da notare che il codice gestisce il caso in cui la rotta passi al middleware l'istanza del modello Project o solo l'ID.

```
$projectParameter = $request->route( param: 'project');

// Se il parametro è già un'istanza del modello Project
if ($projectParameter instanceof Project) {
    $project = $projectParameter;
} else {
    // Se è solo un ID numerico o una stringa, cercolo nel database
    $project = Project::find($projectParameter);
}
```

Figura 19 - EnsureProjectRole, controllo sull'esistenza del progetto

Fatto il controllo dell'esistenza del progetto bisogna inoltre verificare che l'utente autenticato appartenga effettivamente al progetto a cui vuole accedere.

```
$membership = $project->members()
    ->where( column: 'users.id', $user->id)
    ->first();

if (!$membership || !in_array($membership->pivot->role, $roles, strict: true)) {
    abort( code: 403);
}
```

Figura 20 - EnsureProjectRole, controllo sull'associazione dell'utente al progetto

4.2.3 Registrazione e applicazione sulle rotte

I due middleware sono registrati con un alias in bootstrap/app.php e poi applicati alle rotte specificando i ruoli concessi direttamente nella definizione, così da rendere le rotte più esplicite e chiare.

```
$middleware->alias([  
    'system.role' => EnsureSystemRole::class,  
    'project.access' => EnsureProjectRole::class,  
]);
```

Figura 21 - app.php, definizione degli alias per i middleware

In seguito possiamo osservare alcuni esempi di utilizzo all'interno del file dedicato alla gestione delle rotte (routes/web.php).

```
Route::post( uri: '/projects', [ProjectController::class, 'store'])  
    ->name( name: 'projects.store')  
    ->middleware( middleware: 'system.role:admin');  
Route::get( uri: '/projects/{project}', [ProjectController::class, 'show'])  
    ->name( name: 'projects.show')  
    ->middleware( middleware: 'project.access:pm,developer,viewer');
```

Figura 22 - web.php, definizione delle rotte con middleware

Il controllo dei permessi avviene così sia a livello di rotta (il middleware blocca la richiesta con un 403 prima ancora di raggiungere il controller) sia a livello di interfaccia siccome nelle viste Blade le azioni non consentite vengono nascoste in base a variabili come isPm e canEdit definite nel controller. L'utente così, ha un'esperienza d'uso più coerente non visualizzando i pulsanti per eseguire azioni.

4.2.4 Regole critiche

Alcune regole proteggono l'integrità del sistema impedendo stati incoerenti. Alla creazione di un progetto, il creatore viene automaticamente registrato come Project Manager e vengono generate le etichette di sistema. Questo viene fatto per evitare che il progetto non abbia utenti e per fornire agli utenti tutte le etichette di default da poter usare nel progetto. In un gestionale di progetto (come Trello per esempio) i task non hanno per forza assegnato un utente questo perché andrebbe contro il concetto Agile, i task non vengono assegnate ma prese. In un contesto scolastico (caso di questo progetto) ha senso perché è il docente che assegna i task agli allievi. I docenti corrisponderebbero ai Project Manager mentre gli studenti ai Developer.

```
$project = Project::create($request->validated());
// Imposta l'utente che crea il progetto come PM
$project->members()->attach($request->user()->id, ['role' => 'pm']);
// genera le etichette di sistema, createMany itera ogni elemento dell'array
$project->labels()->createMany($this->systemLabels());
```

Figura 23 - ProjectController, creazione di un nuovo progetto

Allo stesso modo, il sistema impedisce di rimuovere l'ultimo amministratore e l'ultimo Project Manager di un progetto, restituendo un errore di validazione 422 anziché eseguire l'operazione.

```
$adminsCount = User::where('sys_role', 'admin')->count();
if ($user->sys_role === 'admin' && $adminsCount === 1) {
    return response()->json(
        ['message' => 'Non puoi eliminare l\'ultimo admin'],
        status: 422
    );
}
```

Figura 24 - AdminController, controllo sull'eliminazione di un utente admin

```
if ($validated['role'] !== 'pm') {
    $checkPm = $project->members()
        ->wherePivot( column: 'role', operator: 'pm')
        ->whereNotIn( column: 'user_id', [$user->id])
        ->get();
    if ($checkPm->isEmpty()) {
        return response()->json(['message' => 'Non puoi cambiare il ruolo all\'ultimo utente PM.'], status: 422);
    }
}
```

Figura 25 - MemberController, controllo sul cambio di ruolo dell'ultimo utente Project Manager

4.2.5 Variante di soluzione

Per il controllo dei permessi, Laravel offre anche il sistema di Policy/Gate. Si è scelto invece di implementare middleware dedicati perché si riesce a specificare in modo chiaro quale rotta è accessibile da un determinato ruolo. Applicare i vincoli sulle rotte rende immediatamente visibile la lettura del file routes/web.php. Inoltre il middleware salva il ruolo risolto tra gli attributi della richiesta (project_role), rendendolo riutilizzabile senza interrogare di nuovo il database.

4.3 Board kanban e spostamento dei task

Ogni progetto possiede una board indipendente formata da colonne ordinate (relazione progetto → colonne → task). L'ordinamento dei task all'interno di una colonna è gestito dal campo position, stessa funzionalità viene adottata per le colonne, che anche loro hanno un proprio campo position per il riordino della board. I task sono collegati agli assegnatari e alle etichette tramite relazioni molti-a-molti (tabelle pivot task_user e label_task), così un task può avere più assegnatari e più etichette.

4.3.1 Creazione di un task

Alla creazione, la posizione del nuovo task viene calcolata come "ultima posizione nella colonna + 1", così il task si accoda in fondo. La sincronizzazione di assegnatari ed etichette usa il metodo sync, che allinea la relazione all'elenco esatto di ID forniti. Se l'utente non seleziona un assegnatario, il form invia comunque l'ID del creatore (gestito lato interfaccia).

```
$task = $project->tasks()->create([
    'title' => $validated['title'],
    'description' => $validated['description'] ?? null,
    'priority' => $validated['priority'],
    'column_id' => $validated['column_id'],
    'due_date' => $validated['due_date'] ?? null,
    'position' => ($project->tasks()
        ->where( column: 'column_id', $validated['column_id'])
        ->max( column: 'position') ?? -1) + 1,
]);

// sync allinea la relazione all'elenco esatto di id (aggiunge/rimuove/mantiene)
$task->assignees()->sync($validated['assignee_ids']);
$task->labels()->sync($validated['label_ids'] ?? []); // se l'utente non seleziona etichette passa array vuoto
```

Figura 26 - TaskController, creazione di un task

4.3.2 Spostamento di un task tra colonne

Lo spostamento è la funzionalità principale della board. Il metodo move riceve la colonna di destinazione e, prima di aggiornarla, la valida assicurandosi che appartenga allo stesso progetto, così un utente non può spostare un task in una colonna di un altro progetto manipolando la richiesta.

```
$validated = $request->validate([
    'column_id' => ['required', 'integer',
        // Controllo sullo spostamento della task solamente all'interno del proprio progetto
        Rule::exists( table: 'project_columns', column: 'id')->where( column: 'project_id', $project->id),
    ],
]);
```

Figura 27 - TaskController, validazione colonna durante lo spostamento

Un singolo spostamento innesca due effetti collaterali: la scrittura di una voce nell'activity log (4.4) e l'invio di una notifica email agli assegnatari (4.5). È proprio questa concatenazione di effetti a rendere il flusso adatto a essere rappresentato con un activity diagram nel capitolo di progettazione.

4.3.3 Variante di soluzione

Lato interfaccia lo spostamento è realizzato con il drag & drop gestito da Alpine.js perché è il gesto atteso in una board kanban e rende l'interazione immediata. La posizione è gestita con numeri interi incrementali anziché, ad esempio, con una lista concatenata. Questa soluzione riscontra una piccola ridondanza in fase di riordino perché si aggiornano più righe per una sola modifica. Questa scelta mantiene le query di lettura semplici (un semplice `orderBy('position')`) e adeguate ai volumi previsti per team di piccole e medie dimensioni. Per uno sviluppo a livello enterprise varrebbe la pena valutare l'approccio gap indexing che consisterebbe di avere intervalli più grandi tra le posizioni.

Esempio di gap indexing:

Task	Position
A	100
B	200
C	300

Se spostiamo C tra A e B:

Task	Position
A	100
C	150
B	200

4.4 Activity log

Ogni azione rilevante viene registrata in un activity log consultabile per progetto. Il modello ActivityLog memorizza i campi fissi (progetto, task, utente autore, tipo di evento e timestamp) e un campo meta che grazie al cast ad array contiene dati di contesto variabili serializzati in JSON. Da notare che la funzione casts non viene chiamata esplicitamente, bensì viene chiamata in automatico quando si legge/scrive l'attributo meta

```
no usages  & christian.arzani
protected function casts(): array
{
    return [
        'meta' => 'array',
    ];
}
```

Figura 28 - Activity log model, cast del campo meta in array

La scelta del campo JSON è motivata dalla gestione molto più semplificata dei dati: uno spostamento ha bisogno di registrare colonna di partenza e di arrivo, un'assegnazione i nomi degli assegnatari, una creazione il solo titolo. Gestire tutti questi dati avrebbe prodotto una tabella sparsa, piena di valori nulli; il campo meta consente invece di adattare i dati a ciascun tipo di evento mantenendo i dati ordinati. I tipi tracciati includono task.created, task.assigned, task.updated, task.moved e task.deleted.

La scrittura avviene direttamente nei controller ogni volta che un'azione rilevante viene eseguita. Ad esempio, nello spostamento di un task.

```
$activity = $project->activityLogs()->create([
    'task_id' => $task->id,
    'user_id' => $request->user()->id,
    'type'    => 'task.moved',
    'meta'    => [
        'task_title' => $task->title,
        'from_column' => $fromColumn,
        'to_column'   => $toColumn,
    ],
]);
```

Figura 29 - TaskController, log della creazione di un task

4.4.1 Variante di soluzione

Un'alternativa sarebbe stata registrare gli eventi automaticamente tramite Model Observer o eventi Eloquent. Si è preferita la scrittura esplicita nei controller per due ragioni: il pieno controllo sui dati di contesto (manipolabili a piacimento per la visualizzazione nella vista) e la leggibilità, dato che l'evento è dichiarato accanto all'azione che lo genera. L'unico fattore negativo di questa soluzione è la ripetizione di codice, che però è accettabile vista la quantità di azioni da inserire nel log.

4.5 Notifiche email e sistema di code

Le notifiche email sfruttano il sistema Notification di Laravel. A ogni evento corrisponde una classe dedicata (TaskAssignment, TaskMove, TaskDueSoon, InviteMember, ...), invocata tramite metodi helper definiti sul modello User che rendono il codice dei controller più espressivo.

```
1 usage  christian.arzani
public function sendTaskMoveNotification(Task $task, User $mover, string $fromColumn, string $toColumn): void
{
    $this->notify(new TaskMove($task, $mover, $fromColumn, $toColumn));
}
```

Figura 30 - Modello User, helper per notificare lo spostamento di un task

4.5.1 Notifiche di scadenza imminente e scheduler

La notifica di scadenza non dipende da un'azione dell'utente ma dal tempo, quindi è affidata a un comando artisan schedulato. Il comando tasks:notify-due-soon seleziona i task in scadenza il giorno successivo, escludendo quelli già completati (colonna is_done) e quelli appartenenti a progetti archiviati, così da non generare invii inutili o errati.

```
$tasks = Task::query()
->whereNotNull( columns: 'due_date') // estrae solo i task con la data di scadenza
->whereDate( column: 'due_date', $tomorrow) // controlla data
// Escludo i task che si trovano nella colonna "completati" (is_done)
->whereHas( relation: 'column', function ($query) {
    $query->where('is_done', false);
})
// Escludo i task dei progetti archiviati (archived_at NULL = attivo)
->whereHas( relation: 'project', function ($query) {
    $query->whereNull('archived_at');
})
->with(['assignees', 'project', 'column'])
->get();

$sent = 0;

foreach ($tasks as $task) {
    foreach ($task->assignees as $assignee) {
        $assignee->notify(new TaskDueSoon($task));
        $sent++;
    }
}
```

Figura 31 - SendDueSoonNotifications, invio notifiche per task in scadenza

Il comando è registrato nello scheduler per essere eseguito una volta al giorno alle 08:00, in background.

```
Schedule::command( command: 'tasks:notify-due-soon')
->dailyAt( time: '08:00')
//Lancia il comando come processo separato non aspetta che finisca
->runInBackground();
```

Figura 32 - console.php, notifiche schedulate

4.5.2 Code e scelta sincrona motivata

Il sistema Notification di Laravel può inviare le email in modo asincrono tramite code (queue), evitando di rallentare la risposta HTTP. Nel contesto di questo progetto si è però scelto, per le notifiche legate alle azioni dirette dell'utente, un invio sincrono protetto da try/catch. Questo rende il comportamento prevedibile in fase di sviluppo e test (l'email parte subito e il suo esito è verificabile immediatamente) e non richiede un worker di coda sempre attivo. Per evitare che un eventuale errore del mail server faccia fallire l'intera operazione, l'invio è incapsulato e l'errore viene soltanto registrato.

```
$emailSent = true;
foreach ($assigneeUsers as $user) {
    try {
        $user->sendTaskAssignmentNotification($task, $request->user());
    } catch (\Throwable $e) {
        $emailSent = false;
        report($e);
    }
}
```

Figura 33 - TaskController, invio notifica incapsulato in un try/catch

In un contesto di produzione la scelta naturale sarebbe spostare l'invio su una coda persistente (driver database o Redis); l'architettura attuale lo consente senza modifiche al codice applicativo, agendo unicamente sulla configurazione, perché le classi di notifica implementano già l'invio asincrono. La scelta di non adottare un approccio con coda persistente è dovuta al fatto che avrei dovuto dedicare del tempo che non avevo a disposizione per risolvere problemi dovuti a questa funzionalità.

4.6 Validazione dei dati e gestione degli errori

Tutta la validazione degli input è centralizzata in classi Form Request, una per ogni operazione che riceve dati. Questo tiene i controller puliti e raccoglie in un unico punto le regole di ciascun form. Le regole coprono tipo, obbligatorietà, range, formato e unicità; i messaggi sono personalizzati in italiano per essere comprensibili all'utente.

```
public function rules(): array
{
    return [
        'name' => 'required|string|max:32|unique:projects,name',
        'description' => 'nullable|string|max:255',
        'priority' => 'required|string|in:low,medium,high',
    ];
}

@arza.exe
public function messages(): array
{
    return [
        'name.required' => 'Il nome del progetto è obbligatorio.',
        'name.string' => 'Il nome del progetto deve essere una stringa di testo.',
        'name.max' => 'Il nome del progetto non può superare i :max caratteri.',
        'description.string' => 'La descrizione deve essere una stringa di testo.',
        'description.max' => 'La descrizione non può superare i :max caratteri.',
        'priority.required' => 'La priorità è obbligatoria.',
        'priority.string' => 'La priorità deve essere una stringa.',
        'priority.in' => 'La priorità deve essere bassa, media o alta.',
    ];
}
```

Figura 34 - StoreTaskRequest, validazione alla creazione di una nuova task

Alcune regole definite sono particolarmente importanti da descrivere, le seguenti sono quelle più particolari: Il formato esadecimale del colore di un'etichetta è validato con un'espressione regolare, l'assegnatario di un task deve obbligatoriamente essere un membro attivo del progetto e la data di scadenza deve essere maggiore di quella di oggi.

```
@christian.arzani
public function rules(): array
{
    return [
        'name' => ['required', 'string', 'max:20'],
        'color' => ['required', 'string', 'regex:/^#[0-9A-Fa-f]{6}$/'],
    ];
}
```

Figura 35 - StoreProjectLabelRequest, validazione alla creazione di nuove etichette

```
'due_date' => ['nullable', 'date', 'after:today'],
'assignee_ids' => ['required', 'array', 'min:1'],
'assignee_ids.*' => [
    'integer',
    Rule::exists( table: 'project_user', column: 'user_id')
    ->where( column: 'project_id', $this->route( param: 'project')->id)
    ->whereNull( column: 'deleted_at'),
],
```

Figura 36 - StoreTaskRequest, validazione alla creazione di un nuovo task

Lato interfaccia, i campi obbligatori sono contrassegnati visivamente, in caso di errore il campo viene evidenziato con il relativo messaggio e i valori già inseriti vengono ripopolati tramite l'helper old() di Blade, così che l'utente non perde quanto digitato. La validazione precede sempre la scrittura, in questo modo in caso di errore nessun dato parziale viene inserito nel database

4.7 Interfaccia grafica

L'interfaccia è stata realizzata con l'ausilio dello strumento AI Claude che ha creato la struttura principale con Blade e Tailwind CSS per lo stile, e Alpine.js per l'interattività leggera (modali, menu a tendina, drag & drop), senza introdurre un framework JavaScript pesante. L'obiettivo è un'interfaccia pulita e immediatamente comprensibile, in cui ogni azione fornisca un riscontro chiaro. Le scelte di usabilità principali sono:

- **Menu laterale persistente** che mostra solo le sezioni effettivamente accessibili in base al ruolo dell'utente, riducendo il rischio di errore e il carico di dati inutile.
- **Navigazione a schede** all'interno del progetto (Board, Dashboard, Membri, Etichette, Activity), che permette di muoversi rapidamente tra le varie sezioni dello stesso progetto mantenendo il contesto.
- **Feedback visivo immediato**: dopo ogni azione viene mostrato un messaggio flash/toast con colorazione (verde per il successo, rosso per l'errore, giallo per gli avvisi).
- **Conferma esplicita per le azioni distruttive o irreversibili** tramite finestra modale: archiviazione di un progetto, eliminazione di task, colonne o membri richiedono una conferma che avverte l'utente
- **Stati visivi chiari** degli elementi interattivi (hover, focus, disabled, loading), così che l'utente percepisca sempre lo stato del sistema.

La coerenza visiva è garantita da un insieme di componenti Blade riutilizzabili (avatar con iniziali, badge di priorità, etichette colorate, pulsanti): definendo questi elementi una volta sola e riutilizzandoli, si evita la duplicazione e si ottiene un aspetto uniforme in tutta l'applicazione. La scelta di Tailwind + Alpine.js è stata fatta per minimizzare il codice di configurazione e mantenere la logica di presentazione vicina al markup, accelerando lo sviluppo senza sacrificare la cura dell'aspetto.

4.8 Pannello amministrativo

La sezione di amministrazione, accessibile solo al ruolo di sistema admin (protetta dal middleware `system.role:admin`), consente la gestione completa degli account: elenco, creazione, modifica del ruolo di sistema ed eliminazione. La creazione di un utente da parte dell'amministratore genera una password temporanea casuale che è costruita garantendo la presenza di almeno una minuscola, una maiuscola e una cifra. La password viene poi comunicata al nuovo utente via email, senza mai essere mostrata in chiaro nell'interfaccia. Anche qui l'invio è protetto da try/catch coerentemente con la strategia descritta in 4.6.

```
$pwd = $this->generatePwd();

$user = User::create([
    'name' => $validated['name'],
    'email' => $validated['email'],
    'sys_role' => $validated['sys_role'] ?? 'user',
    'password' => Hash::make($pwd),
]);

$emailSent = true;
try {
    $user->sendUserRegistrationNotification(auth()->user(), $pwd);
} catch (\Throwable $th) {
    $emailSent = false;
    report($th); // logga l'errore
}
```

Figura 37 - AdminController, gestione password temporanea

Le regole di protezione dell'ultimo amministratore, già illustrate in 4.2, completano questa sezione impedendo che il sistema resti senza alcun account amministrativo.

4.9 Utilizzo dell'AI

Durante lo svolgimento del progetto sono stati utilizzati degli strumenti di intelligenza artificiale, principalmente è stato utilizzato Claude come supporto al lavoro, in modo trasparente e consapevole. L'AI è stata impiegata come assistente alla comprensione e alla risoluzione di problemi, non come sostituto del lavoro di analisi, progettazione e sviluppo.

Gli ambiti principali di utilizzo sono stati quattro. In primo luogo il supporto al debug e alla risoluzione degli errori incontrati durante lo sviluppo, dove l'AI ha aiutato a interpretare i messaggi di errore e a individuare le possibili cause. In secondo luogo la spiegazione di concetti e funzionalità del framework Laravel non trattati in precedenza (p.es notifiche via mail). In terzo luogo la stesura e la revisione della documentazione dal punto di vista della forma, della chiarezza espositiva e dell'organizzazione dei contenuti. Infine, il supporto nella generazione di elementi grafici dell'interfaccia.

Ogni porzione di codice suggerita dall'AI è stata letta, compresa e adattata al contesto specifico dell'applicazione prima di essere integrata, verificandone il funzionamento tramite test. Le decisioni progettuali rilevanti come: l'architettura del sistema, il modello dei dati, la struttura dei ruoli e i permessi sono valutazioni fatte da me, eventualmente è stato discusso con lo strumento la correttezza della mia analisi.

L'utilizzo dell'AI ha avuto quindi l'effetto di accelerare l'apprendimento e di ridurre il tempo speso su problemi tecnici puntuali, lasciando più spazio agli aspetti di progettazione e implementazione. Ne è risultato un metodo di lavoro in cui lo strumento ha agito da supporto alla crescita delle competenze, e non da scorciatoia per aggirarle.

5 Test

I test presenti in questo capitolo sono stati definiti con il supporto di strumenti di intelligenza artificiale, utilizzati come aiuto nella stesura e nella definizione delle casistiche. Ogni test è stato successivamente controllato e adattato manualmente per garantirne la correttezza.

5.1 Protocollo di test

Test Case: Riferimento:	TC-001 REQ-001	Nome:	Registrazione di un nuovo utente con dati validi
Descrizione:	Verifica che un visitatore possa registrare un nuovo account fornendo dati validi e che la password venga salvata come hash.		
Prerequisiti:	• Applicazione installata e database migrato. • Nessun utente registrato con l'email "mario.rossi@example.com". • Utente non autenticato (visitatore).		
Procedura:	1. Aprire la pagina di registrazione (/register). 2. Inserire: nome "Mario Rossi", email "mario.rossi@example.com", password "Password1", conferma password "Password1". 3. Cliccare il pulsante "Registrati". 4. Eseguire la query SQL: SELECT name, email, password FROM users WHERE email = 'mario.rossi@example.com';		
Risultati attesi:	L'utente viene creato e autenticato, con reindirizzamento alla dashboard. Nel DB l'utente esiste con ruolo di sistema "user" e il campo password contiene un hash bcrypt (prefisso \$2y\$), non la password in chiaro.		

Test Case: Riferimento:	TC-002 REQ-001	Nome:	Registrazione con email già esistente
Descrizione:	Verifica il vincolo di unicità dell'email in fase di registrazione.		
Prerequisiti:	• Esiste già un utente registrato con l'email "mario.rossi@example.com". • Utente non autenticato.		
Procedura:	1. Aprire la pagina di registrazione (/register). 2. Inserire nome "Mario Bianchi", email "mario.rossi@example.com", password valida "Password1" e relativa conferma. 3. Cliccare il pulsante "Registrati".		
Risultati attesi:	La registrazione viene rifiutata. Viene mostrato un messaggio di errore sul campo email che indica che l'email è già in uso. Nessun nuovo utente viene creato nel DB.		

Test Case: Riferimento:	TC-003 REQ-001	Nome:	Registrazione con password non conforme ai criteri
Descrizione:	Verifica le regole sulla password (min. 8 caratteri, almeno 1 maiuscola e 1 numero) e la conferma password.		
Prerequisiti:	• Utente non autenticato.		
Procedura:	1. Aprire la pagina di registrazione (/register). 2. Caso A: inserire password "abc" (troppo corta) e confermarla, poi inviare. 3. Caso B: inserire password "password" (senza maiuscola né numero) e confermarla, poi inviare. 4. Caso C: inserire password "Password1" e conferma password "Password2" (non corrispondenti), poi inviare.		
Risultati attesi:	In tutti i casi la registrazione viene rifiutata e viene mostrato un messaggio di errore specifico relativo al campo password. Nessun utente viene creato.		

Test Case: Riferimento:	TC-004 REQ-001	Nome:	Login con credenziali corrette e logout
Descrizione:	Verifica l'autenticazione con credenziali valide, la gestione della sessione e il logout.		
Prerequisiti:	• Esiste un utente "mario.rossi@example.com" con password "Password1". • Utente non autenticato.		
Procedura:	1. Aprire la pagina di login (/login). 2. Inserire email "mario.rossi@example.com" e password "Password1". 3. Cliccare "Accedi". 4. Verificare l'accesso a una rotta protetta (es. /projects). 5. Cliccare il pulsante "Logout". 6. Tentare nuovamente di accedere alla rotta protetta /projects.		
Risultati attesi:	Dopo il login l'utente accede alla dashboard e può visualizzare le rotte protette. Dopo il logout la sessione viene chiusa e il tentativo di accesso a /projects reindirizza alla pagina di login.		

Test Case: Riferimento:	TC-005 REQ-001	Nome:	Login con credenziali errate
Descrizione:	Verifica che l'accesso venga negato con password errata e che non vengano esposti dettagli sensibili.		
Prerequisiti:	• Esiste un utente "mario.rossi@example.com" con password "Password1". • Utente non autenticato.		
Procedura:	1. Aprire la pagina di login (/login). 2. Inserire email "mario.rossi@example.com" e password errata "Sbagliata9". 3. Cliccare "Accedi".		
Risultati attesi:	L'accesso viene negato. Viene mostrato un messaggio di errore generico sulle credenziali non valide (senza distinguere se l'errore riguarda email o password). L'utente non viene autenticato.		

Test Case: Riferimento:	TC-006 REQ-001 / REQ-NF-005	Nome:	Accesso a rotta protetta senza autenticazione (middleware auth)
Descrizione:	Verifica che il middleware di autenticazione protegga le rotte riservate agli utenti loggati.		
Prerequisiti:	• Utente non autenticato. • Esiste almeno un progetto con id noto (es. id = 1).		
Procedura:	1. Da browser non autenticato, digitare direttamente nell'URL la rotta /projects/1. 2. Inviare la richiesta.		
Risultati attesi:	L'accesso viene bloccato dal middleware auth e l'utente viene reindirizzato alla pagina di login senza visualizzare il contenuto protetto.		

Test Case: Riferimento:	TC-007 REQ-002	Nome:	L'Admin accede alla pagina di amministrazione utenti
Descrizione:	Verifica che la pagina di gestione utenti sia accessibile al solo ruolo di sistema Admin.		
Prerequisiti:	• Esiste un utente con ruolo di sistema "admin". • Admin autenticato.		
Procedura:	1. Effettuare il login come Admin. 2. Accedere alla sezione di amministrazione utenti (es. /admin/users).		
Risultati attesi:	La pagina di amministrazione utenti viene visualizzata con l'elenco di tutti gli account e le relative azioni di gestione (CRUD).		

Test Case: Riferimento:	TC-008 REQ-002 / REQ-NF-005	Nome:	L'utente standard non può accedere all'amministrazione utenti
Descrizione:	Verifica il controllo di autorizzazione per ruolo: un User non deve accedere alle funzioni di Admin.		
Prerequisiti:	• Esiste un utente con ruolo di sistema "user". • User autenticato.		
Procedura:	1. Effettuare il login come User standard. 2. Tentare di accedere alla rotta /admin/users digitandola nell'URL.		
Risultati attesi:	L'accesso viene negato con una pagina di errore HTTP 403 (Forbidden). La pagina di amministrazione non viene mostrata.		

Test Case: Riferimento:	TC-009 REQ-002	Nome:	L'Admin modifica il ruolo di sistema di un utente
Descrizione:	Verifica che l'Admin possa cambiare il ruolo di sistema di un altro utente (es. da User ad Admin).		
Prerequisiti:	• Admin autenticato. • Esiste un utente "luca.verdi@example.com" con ruolo di sistema "user".		
Procedura:	1. Accedere alla pagina di amministrazione utenti. 2. Selezionare l'utente "luca.verdi@example.com". 3. Cambiare il ruolo di sistema in "admin" e salvare. 4. Eseguire la query SQL: SELECT email, role FROM users WHERE email = 'luca.verdi@example.com';		
Risultati attesi:	Il ruolo dell'utente viene aggiornato. Nel DB il campo role risulta "admin" e l'utente acquisisce i relativi permessi.		

Test Case: Riferimento:	TC-010 REQ-003	Nome:	Creazione di un progetto con dati validi
Descrizione:	Verifica la creazione di un progetto con tutti i campi validi e l'assegnazione del creatore come Project Manager.		
Prerequisiti:	• Utente autenticato (User o Admin).		
Procedura:	1. Accedere alla sezione progetti e cliccare "Nuovo progetto". 2. Inserire titolo "Progetto Alfa", descrizione "Descrizione di prova", priorità "alta", data di scadenza futura, stato "attivo". 3. Cliccare "Salva". 4. Eseguire la query SQL: SELECT title, status, priority FROM projects WHERE title = 'Progetto Alfa';		
Risultati attesi:	Il progetto viene creato e compare nella lista. Nel DB il progetto esiste con stato "attivo". Il creatore risulta membro del progetto con ruolo Project Manager e viene creata la board con le colonne di default.		

Test Case: Riferimento:	TC-011 REQ-003 / REQ-NF-001	Nome:	Creazione progetto con titolo duplicato
Descrizione:	Verifica il vincolo di unicità del titolo del progetto.		
Prerequisiti:	• Esiste già un progetto con titolo "Progetto Alfa". • Utente autenticato.		
Procedura:	1. Cliccare "Nuovo progetto". 2. Inserire titolo "Progetto Alfa" (già esistente) e gli altri campi validi. 3. Cliccare "Salva".		
Risultati attesi:	La creazione viene rifiutata. Viene mostrato un messaggio di errore sul campo titolo che indica la duplicazione. I dati già inseriti rimangono nel form. Nessun progetto duplicato viene creato.		

Test Case: Riferimento:	TC-012 REQ-003	Nome:	Archiviazione progetto con conferma (soft delete)
Descrizione:	Verifica che il cambio di stato in "archiviato" mostri un avviso e renda il progetto di sola lettura.		
Prerequisiti:	• Esiste un progetto "Progetto Alfa" in stato "attivo". • Utente autenticato come Project Manager o Admin del progetto.		
Procedura:	1. Aprire il dettaglio del progetto "Progetto Alfa". 2. Cliccare l'azione "Archivia progetto". 3. Verificare la comparsa del messaggio di conferma con avviso di irreversibilità. 4. Confermare l'archiviazione. 5. Eseguire la query SQL: SELECT title, status FROM projects WHERE title = 'Progetto Alfa';		
Risultati attesi:	Prima dell'archiviazione viene mostrato un modal di conferma con avviso di irreversibilità. Dopo la conferma lo stato nel DB diventa "archiviato" e il progetto risulta di sola lettura (le azioni di modifica sono disattivate).		

Test Case: Riferimento:	TC-013 REQ-003	Nome:	Modifica di un progetto archiviato non consentita
Descrizione:	Verifica che un progetto archiviato non possa più essere modificato ma rimanga visualizzabile.		
Prerequisiti:	• Esiste un progetto "Progetto Alfa" in stato "archiviato". • Utente autenticato come Project Manager del progetto.		
Procedura:	1. Aprire il dettaglio del progetto archiviato "Progetto Alfa". 2. Tentare di accedere alla funzione di modifica del progetto (anche forzando la rotta di update via URL/POST).		
Risultati attesi:	Il progetto resta visualizzabile in sola lettura. Qualsiasi tentativo di modifica viene bloccato (azioni nascoste in interfaccia e richiesta respinta lato server). I dati del progetto non vengono alterati.		

Test Case: Riferimento:	TC-014 REQ-003 / REQ-NF-005	Nome:	Un Viewer non può eliminare o modificare un progetto
Descrizione:	Verifica che modifica ed eliminazione siano consentite solo a Project Manager o Admin.		
Prerequisiti:	• Esiste un progetto "Progetto Alfa" in stato "attivo". • Utente autenticato come membro del progetto con ruolo Viewer.		
Procedura:	1. Aprire il dettaglio del progetto come Viewer. 2. Verificare la presenza/assenza dei pulsanti di modifica ed eliminazione. 3. Tentare di richiamare direttamente la rotta di eliminazione del progetto via URL.		
Risultati attesi:	I pulsanti di modifica ed eliminazione non sono visibili al Viewer. Il tentativo di accesso diretto alla rotta restituisce un errore HTTP 403 e il progetto non viene modificato né eliminato.		

Test Case: Riferimento:	TC-015 REQ-004	Nome:	Il Project Manager invita un nuovo membro assegnando un ruolo
Descrizione:	Verifica l'invito di un utente registrato come membro del progetto con un ruolo di progetto.		
Prerequisiti:	• Esiste un progetto "Progetto Alfa". • Esiste un utente registrato "luca.verdi@example.com" non ancora membro del progetto. • Utente autenticato come Project Manager del progetto.		
Procedura:	1. Aprire la sezione "Membri" del progetto. 2. Cliccare "Invita membro". 3. Selezionare l'utente "luca.verdi@example.com" e assegnare il ruolo "Developer". 4. Confermare l'invito. 5. Verificare la lista membri del progetto.		
Risultati attesi:	L'utente viene aggiunto come membro del progetto con ruolo Developer e compare nella lista membri con il ruolo corretto.		

Test Case: Riferimento:	TC-016 REQ-004	Nome:	Il Project Manager modifica il ruolo di un membro e lo rimuove
Descrizione:	Verifica la modifica del ruolo di progetto di un membro e la sua rimozione.		
Prerequisiti:	- Esiste un progetto "Progetto Alfa" con membro "luca.verdi@example.com" con ruolo Developer. - Utente autenticato come Project Manager del progetto.		
Procedura:	- Aprire la sezione "Membri" del progetto. - Modificare il ruolo di "luca.verdi@example.com" da Developer a Viewer e salvare. - Verificare l'aggiornamento del ruolo nella lista. - Rimuovere il membro "luca.verdi@example.com" dal progetto e confermare.		
Risultati attesi:	Il ruolo del membro viene aggiornato a Viewer; dopo la rimozione il membro non compare più nella lista membri e perde l'accesso al progetto.		

Test Case: Riferimento:	TC-017 REQ-005	Nome:	Il Developer può operare sui task ma non gestire i membri
Descrizione:	Verifica i permessi del ruolo Developer (operatività sui task, commenti, etichette) e l'impossibilità di gestire membri/colonne.		
Prerequisiti:	- Esiste un progetto "Progetto Alfa" con almeno una colonna e un task. - Utente autenticato come membro del progetto con ruolo Developer.		
Procedura:	- Aprire la board del progetto come Developer. - Verificare la possibilità di creare/modificare/spostare un task e aggiungere un commento. - Verificare la presenza/assenza delle azioni di gestione membri e di gestione colonne.		
Risultati attesi:	Il Developer può creare, modificare e spostare i task e aggiungere commenti/etichette. Le funzioni di gestione membri e di gestione colonne non sono accessibili (azioni nascoste e rotte protette).		

Test Case: Riferimento:	TC-018 REQ-005	Nome:	Il Viewer ha accesso in sola lettura
Descrizione:	Verifica che il ruolo Viewer possa solo visualizzare progetto, board, task, commenti, log e dashboard.		
Prerequisiti:	- Esiste un progetto "Progetto Alfa" con board, task e commenti. - Utente autenticato come membro del progetto con ruolo Viewer.		
Procedura:	- Aprire il progetto come Viewer. - Navigare board, dettaglio task, commenti, activity log e dashboard. - Verificare l'assenza di qualsiasi pulsante di creazione/modifica/spostamento. - Tentare di richiamare via URL una rotta di creazione task.		
Risultati attesi:	Il Viewer visualizza tutti i contenuti del progetto ma non vede alcuna azione di scrittura. Il tentativo di accesso diretto a una rotta di scrittura restituisce HTTP 403.		

Test Case: Riferimento:	TC-019 REQ-006	Nome:	Colonne di default selezionabili con la creazione di un nuovo progetto
Descrizione:	Verifica che alla creazione di una colonna vengano visualizzate le colonne di default previste.		
Prerequisiti:	• Utente autenticato. • Creazione di un nuovo progetto in corso. • Utente autenticato come Project Manager del progetto.		
Procedura:	1. Creare un nuovo progetto "Progetto Beta". 2. Aprire la board del progetto appena creato. 3. Creare una colonna 4. Verificare le tipologie di colonne selezionabili.		
Risultati attesi:	Il form mostra la lista di colonne di default selezionabili dall'utente: "Backlog", "Da fare", "In corso", "In revisione", "Testing", "Completato", nell'ordine corretto.		

Test Case: Riferimento:	TC-020 REQ-006	Nome:	Il Project Manager aggiunge, riordina ed elimina una colonna
Descrizione:	Verifica la gestione delle colonne kanban da parte del Project Manager.		
Prerequisiti:	• Esiste un progetto "Progetto Beta" con le colonne di default e senza task. • Utente autenticato come Project Manager del progetto.		
Procedura:	1. Aprire la board del progetto. 2. Aggiungere una nuova colonna "In attesa". 3. Riordinare la colonna "In attesa" spostandola dopo "Da fare". 4. Eliminare la colonna "In attesa" (vuota) e confermare.		
Risultati attesi:	La nuova colonna viene creata, riordinata nella posizione richiesta ed eliminata correttamente; la board riflette le modifiche in modo persistente.		

Test Case: Riferimento:	TC-021 REQ-006	Nome:	Eliminazione di una colonna contenente task con spostamento
Descrizione:	Verifica che l'eliminazione di una colonna con task richieda conferma e lo spostamento dei task in un'altra colonna.		
Prerequisiti:	• Esiste un progetto con la colonna "In corso" contenente almeno 1 task. • Utente autenticato come Project Manager del progetto.		
Procedura:	1. Aprire la board del progetto. 2. Avviare l'eliminazione della colonna "In corso". 3. Nel dialog di conferma, selezionare la colonna di destinazione "Da fare" per i task. 4. Confermare l'operazione. 5. Verificare la colonna "Da fare".		
Risultati attesi:	Viene richiesta conferma con scelta della colonna di destinazione. Dopo la conferma la colonna "In corso" viene eliminata e i suoi task risultano spostati nella colonna "Da fare" senza perdita di dati.		

Test Case: Riferimento:	TC-022 REQ-007	Nome:	Creazione task con assegnatario di default
Descrizione:	Verifica la creazione di un task e l'assegnazione automatica al creatore quando non viene scelto un assegnatario.		
Prerequisiti:	- Esiste un progetto con almeno una colonna. - Utente autenticato come Developer o Project Manager del progetto (es. "mario.rossi@example.com").		
Procedura:	- Aprire la board e cliccare "Nuovo task" nella colonna "Da fare". - Inserire titolo "Implementare login", priorità "alta", senza selezionare un assegnatario. - Salvare il task. - Eseguire la query SQL: <code>SELECT title, assignee_id, column_id FROM tasks WHERE title = 'Implementare login';</code>		
Risultati attesi:	Il task viene creato nella colonna "Da fare". Nel DB il campo assignee_id corrisponde all'utente creatore (Mario Rossi), poiché non è stato selezionato alcun assegnatario.		

Test Case: Riferimento:	TC-023 REQ-007	Nome:	Creazione task con assegnatario non membro del progetto
Descrizione:	Verifica che l'assegnatario di un task debba essere un membro del progetto.		
Prerequisiti:	- Esiste un progetto "Progetto Alfa" con membri noti. - Esiste un utente registrato non membro del progetto. - Utente autenticato come Project Manager del progetto.		
Procedura:	- Aprire la creazione di un nuovo task. - Tentare di assegnare il task a un utente che non è membro del progetto (es. forzando l'id nel form/POST). - Inviare il form.		
Risultati attesi:	La creazione viene rifiutata con un messaggio di errore sul campo assegnatario. Il task non viene salvato con un assegnatario non appartenente al progetto.		

Test Case: Riferimento:	TC-024 REQ-007	Nome:	Modifica ed eliminazione di un task con conferma
Descrizione:	Verifica la modifica dei campi di un task e l'eliminazione con conferma.		
Prerequisiti:	- Esiste un task "Implementare login" in un progetto. - Utente autenticato come Developer o Project Manager del progetto.		
Procedura:	- Aprire il dettaglio del task "Implementare login". - Modificare la descrizione e la priorità in "media", quindi salvare. - Avviare l'eliminazione del task. - Confermare nel dialog di conferma.		
Risultati attesi:	Le modifiche vengono salvate e visualizzate correttamente. L'eliminazione richiede conferma e, dopo conferma, il task non compare più nella board né nel DB.		

Test Case: Riferimento:	TC-025 REQ-008	Nome:	Spostamento di un task tra colonne dall'interfaccia
Descrizione:	Verifica che lo spostamento di un task tra colonne aggiorni la colonna di appartenenza e l'ordine.		
Prerequisiti:	• Esiste un progetto con le colonne "Da fare" e "In corso". • Esiste un task "Implementare login" nella colonna "Da fare". • Utente autenticato come Developer o Project Manager del progetto.		
Procedura:	1. Aprire la board del progetto. 2. Spostare il task "Implementare login" dalla colonna "Da fare" alla colonna "In corso" tramite l'interfaccia (drag&drop o select). 3. Eseguire la query SQL: SELECT title, column_id FROM tasks WHERE title = 'Implementare login';		
Risultati attesi:	Il task appare nella colonna "In corso". Nel DB il campo column_id viene aggiornato con l'id della colonna "In corso" e l'ordine all'interno della colonna è coerente.		

Test Case: Riferimento:	TC-026 REQ-009	Nome:	Presenza delle etichette di default e loro non modificabilità
Descrizione:	Verifica che le etichette di default siano presenti dopo il seeding e non possano essere modificate o eliminate.		
Prerequisiti:	• Database migrato e seeders eseguiti. • Utente autenticato come Project Manager di un progetto.		
Procedura:	1. Aprire la sezione gestione etichette di un progetto. 2. Verificare l'elenco delle etichette di default. 3. Tentare di modificare o eliminare l'etichetta "BACKEND" (anche forzando la rotta via URL).		
Risultati attesi:	Sono presenti le etichette di default DIAGRAMMA, ANALISI, BACKEND, FRONTEND, DOCUMENTAZIONE, TEST. Le azioni di modifica/eliminazione non sono disponibili e il tentativo forzato viene respinto; le etichette di default restano invariate.		

Test Case: Riferimento:	TC-027 REQ-009 / REQ-NF-001	Nome:	Creazione etichetta personalizzata con validazione del colore
Descrizione:	Verifica la creazione di un'etichetta personalizzata e la validazione del formato colore esadecimale.		
Prerequisiti:	• Esiste un progetto "Progetto Alfa". • Utente autenticato come Developer o Project Manager del progetto.		
Procedura:	1. Aprire la gestione etichette del progetto. 2. Caso A: creare un'etichetta "Urgente" con colore "#FF0000" e salvare. 3. Caso B: tentare di creare un'etichetta con colore non valido "rosso" e salvare. 4. Associare l'etichetta "Urgente" a un task.		
Risultati attesi:	Caso A: l'etichetta "Urgente" viene creata con il colore indicato. Caso B: la creazione viene rifiutata con messaggio di errore sul formato colore (deve essere esadecimale). L'etichetta valida può essere associata a uno o più task.		

Test Case: Riferimento:	TC-028 REQ-010	Nome:	Gestione checklist di sotto-attività di un task
Descrizione:	Verifica l'aggiunta, il completamento e il conteggio delle sotto-attività di un task.		
Prerequisiti:	- Esiste un task "Implementare login" in un progetto. - Utente autenticato come Developer o Project Manager del progetto.		
Procedura:	- Aprire il dettaglio del task "Implementare login". - Aggiungere due sotto-attività: "Creare form" e "Validare credenziali". - Tentare di aggiungere una sotto-attività senza titolo. - Contrassegnare "Creare form" come completata. - Verificare il conteggio mostrato dal task.		
Risultati attesi:	Le sotto-attività con titolo vengono aggiunte; quella senza titolo viene rifiutata (titolo obbligatorio). Dopo aver completato una sotto-attività il task mostra il conteggio "1 / 2".		

Test Case: Riferimento:	TC-029 REQ-011 / REQ-013	Nome:	Commento con menzione @username genera notifica
Descrizione:	Verifica che una menzione nel commento/descrizione del task generi una notifica all'utente menzionato.		
Prerequisiti:	- Esiste un progetto "Progetto Alfa" con membri "mario.rossi" e "luca.verdi". - Mailer configurato (log o SMTP di test). - Utente "mario.rossi" autenticato e membro del progetto.		
Procedura:	- Aprire il dettaglio di un task del progetto. - Inserire nella descrizione/commento il testo: "@luca.verdi puoi verificare?". - Salvare il commento. - Verificare le notifiche/email generate per l'utente menzionato.		
Risultati attesi:	Il commento viene salvato con la menzione riconosciuta. Per l'utente "luca.verdi" viene generata una notifica via email relativa alla menzione.		

Test Case: Riferimento:	TC-030 REQ-012 / criterio 224	Nome:	Registrazione degli eventi rilevanti nell'activity log
Descrizione:	Verifica che le azioni rilevanti (creazione task, cambio di stato, assegnazione) vengano registrate nel log di progetto con i dati di contesto.		
Prerequisiti:	- Esiste un progetto "Progetto Alfa" con almeno due colonne e un membro assegnatario. - Utente autenticato come Developer o Project Manager del progetto.		
Procedura:	- Creare un nuovo task "Task log test". - Assegnare il task a un membro del progetto. - Spostare il task in un'altra colonna (cambio di stato). - Aprire la sezione activity log del progetto. - Eseguire la query SQL: SELECT event_type, user_id, task_id, created_at FROM activity_logs WHERE project_id = (SELECT id FROM projects WHERE title = 'Progetto Alfa') ORDER BY created_at;		
Risultati attesi:	Nell'activity log compaiono le voci relative a creazione task, assegnazione e cambio di stato. Ogni voce contiene utente autore, tipo di evento, task coinvolto, timestamp e dati di contesto, coerenti con quanto registrato nel DB.		

Test Case: Riferimento:	TC-031 REQ-012	Nome:	Activity log filtrabile e accessibile a tutti i membri
Descrizione:	Verifica che il log sia visualizzabile per progetto e accessibile a tutti i membri, incluso il Viewer.		
Prerequisiti:	• Esiste un progetto "Progetto Alfa" con voci nel log. • Utente autenticato come membro del progetto con ruolo Viewer.		
Procedura:	1. Accedere al progetto come Viewer. 2. Aprire la sezione activity log del progetto. 3. Applicare il filtro per progetto/visualizzazione.		
Risultati attesi:	Il Viewer può visualizzare l'activity log del progetto in sola lettura; le voci mostrate appartengono esclusivamente al progetto selezionato.		

Test Case: Riferimento:	TC-032 REQ-013	Nome:	Notifica email all'assegnazione di un task
Descrizione:	Verifica l'invio di una notifica email all'utente quando gli viene assegnato un task.		
Prerequisiti:	• Mailer configurato (driver log o SMTP di test). • Esiste un progetto con il membro "luca.verdi@example.com". • Utente autenticato come Project Manager del progetto.		
Procedura:	1. Creare o modificare un task assegnandolo a "luca.verdi@example.com". 2. Salvare. 3. Verificare la coda/log delle email inviate (es. storage/logs/laravel.log o Mailtrap).		
Risultati attesi:	Viene generata e inviata un'email di notifica all'assegnatario "luca.verdi@example.com", contenente il riferimento al task assegnato.		

Test Case: Riferimento:	TC-033 REQ-013	Nome:	Notifica email all'avvicinarsi della scadenza di un task
Descrizione:	Verifica l'invio della notifica di scadenza imminente (es. 24h prima) tramite il task scheduler.		
Prerequisiti:	• Mailer configurato. • Esiste un task con assegnatario e data di scadenza fissata a circa 24 ore dal momento corrente. • Scheduler/comando di notifica scadenze configurato.		
Procedura:	1. Predisporre un task con scadenza entro le successive 24 ore. 2. Eseguire manualmente il comando schedulato di controllo scadenze (es. php artisan schedule:run oppure il comando dedicato). 3. Verificare il log/coda delle email.		
Risultati attesi:	Viene inviata un'email di notifica all'assegnatario del task riguardante l'avvicinarsi della scadenza. Il task scaduto/non interessato non genera invii errati.		

Test Case: Riferimento:	TC-034 REQ-013	Nome:	Notifica email al cambio di stato di un task assegnato
Descrizione:	Verifica l'invio di notifica all'assegnatario quando il task cambia colonna/stato.		
Prerequisiti:	• Mailer configurato. • Esiste un task assegnato a "luca.verdi@example.com". • Utente autenticato (diverso dall'assegnatario) con permesso di spostare il task.		
Procedura:	1. Spostare il task assegnato a "luca.verdi@example.com" in una colonna diversa. 2. Verificare il log/coda delle email inviate.		
Risultati attesi:	Viene inviata un'email di notifica all'assegnatario relativa al cambiamento di stato (colonna) del task.		

Test Case: Riferimento:	TC-035 REQ-014	Nome:	Dashboard di progetto con statistiche corrette
Descrizione:	Verifica che la dashboard mostri percentuale di completamento, conteggio task per stato, task scaduti e task per membro.		
Prerequisiti:	• Esiste un progetto "Progetto Alfa" con un insieme noto di task: alcuni completati, alcuni scaduti, distribuiti su più colonne e assegnati a membri diversi.		
Procedura:	1. Aprire la dashboard del progetto "Progetto Alfa". 2. Confrontare la percentuale di completamento con il rapporto task completati / totali noti. 3. Verificare il numero di task per ciascuna colonna/stato. 4. Verificare l'elenco dei task scaduti (scadenza < oggi e non completati). 5. Verificare il numero di task assegnati per ciascun membro.		
Risultati attesi:	La dashboard mostra valori coerenti con i dati noti: percentuale di completamento corretta, conteggi per stato corretti, elenco task scaduti corretto e numero di task per membro corretto.		

Test Case: Riferimento:	TC-036 REQ-014 / REQ-NF-004	Nome:	Dashboard con progetto senza task (caso limite)
Descrizione:	Verifica il comportamento della dashboard quando il progetto non ha task.		
Prerequisiti:	• Esiste un progetto "Progetto Vuoto" senza alcun task. • Utente autenticato come membro del progetto.		
Procedura:	1. Aprire la dashboard del progetto "Progetto Vuoto".		
Risultati attesi:	La dashboard viene visualizzata senza errori: percentuale di completamento mostrata come 0% (senza divisione per zero), conteggi a 0 ed elenco task scaduti vuoto.		

Test Case: Riferimento:	TC-037 REQ-015	Nome:	Esportazione del report PDF della dashboard
Descrizione:	Verifica la generazione e il download del report PDF di riepilogo del progetto.		
Prerequisiti:	• Esiste un progetto "Progetto Alfa" con task e statistiche. • Utente autenticato come membro del progetto.		
Procedura:	1. Aprire la dashboard del progetto "Progetto Alfa". 2. Cliccare il pulsante "Esporta report PDF". 3. Aprire il file PDF scaricato.		
Risultati attesi:	Viene generato e scaricato un file PDF contenente i metadati del progetto, le statistiche della dashboard e la lista dei task con il relativo stato; il documento è leggibile e completo.		

Test Case: Riferimento:	TC-038 REQ-NF-001 / criterio 194	Nome:	Validazione form con ripopolamento dei dati e nessun salvataggio parziale
Descrizione:	Verifica la validazione lato server, il ripopolamento dei dati inseriti in caso di errore e l'assenza di salvataggi parziali.		
Prerequisiti:	• Utente autenticato. • Form di creazione progetto disponibile.		
Procedura:	1. Aprire il form di creazione progetto. 2. Compilare la descrizione e la priorità ma lasciare vuoto il campo obbligatorio "titolo". 3. Inviare il form. 4. Verificare il form ritornato e il database.		
Risultati attesi:	Il form viene rifiutato lato server: il campo titolo è evidenziato con messaggio specifico, i campi obbligatori sono contrassegnati visivamente, i valori già inseriti (descrizione, priorità) vengono ripopolati. Nel DB non viene salvato alcun progetto parziale.		

Test Case: Riferimento:	TC-039 REQ-NF-002 / criterio 164	Nome:	Pagine di errore personalizzate 403 e 404
Descrizione:	Verifica la presenza di pagine di errore personalizzate e comprensibili, senza esporre dettagli tecnici.		
Prerequisiti:	• Applicazione in esecuzione. • Utente autenticato come User standard.		
Procedura:	1. Richiamare una rotta inesistente (es. /rotta-che-non-esiste) per ottenere un 404. 2. Richiamare una rotta riservata all'Admin come User per ottenere un 403. 3. Osservare le pagine di errore mostrate.		
Risultati attesi:	Vengono mostrate pagine di errore personalizzate per HTTP 404 e 403, con messaggi comprensibili all'utente e senza esporre stack trace o dettagli tecnici interni.		

Test Case: Riferimento:	TC-040 REQ-NF-003 / criterio 146	Nome:	Feedback visivo e conferma per azioni distruttive
Descrizione:	Verifica il feedback visivo (flash/toast) dopo le azioni e la conferma esplicita per le azioni irreversibili.		
Prerequisiti:	• Esiste un progetto con un task. • Utente autenticato come Project Manager del progetto.		
Procedura:	1. Eseguire un'azione di successo (es. creare un task) e osservare il messaggio di feedback. 2. Avviare un'azione distruttiva (es. eliminare un task) e osservare la richiesta di conferma. 3. Annullare e poi confermare l'azione.		
Risultati attesi:	Dopo le azioni viene mostrato un messaggio di feedback con colorazione semantica (verde successo / rosso errore). Le azioni distruttive richiedono una conferma esplicita tramite modal; annullando, l'azione non viene eseguita.		

Test Case: Riferimento:	TC-041 REQ-NF-005 / criterio 164	Nome:	Protezione CSRF sui form
Descrizione:	Verifica che i form siano protetti dal token CSRF di Laravel.		
Prerequisiti:	• Utente autenticato. • Strumento per inviare richieste HTTP (es. browser/devtools o client REST).		
Procedura:	1. Inviare una richiesta POST a una rotta di creazione (es. creazione task) omettendo o alterando il token CSRF. 2. Osservare la risposta del server.		
Risultati attesi:	La richiesta priva di token CSRF valido viene respinta dal server (errore HTTP 419/Page Expired). L'operazione non viene eseguita.		

Test Case: Riferimento:	TC-042 REQ-NF-005 / criterio 237	Nome:	Protezione contro XSS nell'output
Descrizione:	Verifica che l'input utente contenente codice HTML/script venga mostrato in modo sicuro (escape automatico).		
Prerequisiti:	• Utente autenticato come Developer di un progetto.		
Procedura:	1. Creare un task inserendo come titolo la stringa: <code><script>alert('XSS')</script></code>. 2. Salvare e aprire la board/dettaglio del task. 3. Osservare il rendering del titolo nella pagina.		
Risultati attesi:	Il contenuto viene mostrato come testo letterale (escape automatico tramite la sintassi Blade <code>{{ }}</code>); lo script non viene eseguito e non compare alcun popup di alert.		

5.2 Risultati test

Test Case	Riferimento	Descrizione	Risultato effettivo
TC-001	REQ-001	Nome: Registrazione di un nuovo utente con dati validi Risultato atteso: Un visitatore registra un account con dati validi; l'utente viene creato e autenticato con ruolo di sistema "user" e la password è salvata come hash bcrypt (prefisso \$2y\$), non in chiaro.	SUPERATO – L'utente è stato creato e autenticato con ruolo "user"; nel DB la password risulta salvata come hash bcrypt (\$2y\$). Esito conforme al risultato atteso. PASS Tests\Feature\AuthenticationTest ✓ valid data registration Tests: 1 passed (6 assertions) Duration: 0.62s
TC-002	REQ-001	Nome: Registrazione con email già esistente Risultato atteso: La registrazione con un'email già in uso viene rifiutata; viene mostrato un errore sul campo email e nessun nuovo utente viene creato (vincolo di unicità).	SUPERATO – La registrazione è stata rifiutata con messaggio di errore sul campo email; nessun nuovo utente è stato creato. Esito conforme al risultato atteso. PASS Tests\Feature\AuthenticationTest ✓ registration with duplicate email Tests: 1 passed (3 assertions) Duration: 0.55s
TC-003	REQ-001	Nome: Registrazione con password non conforme ai criteri Risultato atteso: Password troppo corta, senza maiuscola/numero o conferma non corrispondente: la registrazione viene rifiutata con messaggio specifico sul campo password e nessun utente creato.	SUPERATO – In tutti i casi (A, B, C) la registrazione è stata rifiutata con messaggio specifico sul campo password; nessun utente è stato creato. Esito conforme al risultato atteso. PASS Tests\Feature\AuthenticationTest ✓ short password ✓ password without upper or number ✓ password confirmation mismatch Tests: 3 passed (9 assertions) Duration: 0.62s
TC-004	REQ-001	Nome: Login con credenziali corrette e logout Risultato atteso: Con credenziali valide l'utente accede alla dashboard e alle rotte protette; dopo il logout la sessione viene chiusa e l'accesso a /projects reindirizza al login.	SUPERATO – Login e accesso alle rotte protette riusciti; dopo il logout la sessione è stata chiusa e l'accesso a /projects ha reindirizzato al login. Esito conforme al risultato atteso. PASS Tests\Feature\AuthenticationTest ✓ login and logout Tests: 1 passed (8 assertions) Duration: 0.60s

Test Case	Riferimento	Descrizione	Risultato effettivo
TC-005	REQ-001	Nome: Login con credenziali errate Risultato atteso: Con password errata l'accesso viene negato; viene mostrato un messaggio generico sulle credenziali non valide senza esporre dettagli sensibili. L'utente non viene autenticato.	SUPERATO – L'accesso è stato negato con messaggio generico sulle credenziali; nessun dettaglio sensibile esposto e utente non autenticato. Esito conforme al risultato atteso. PASS Tests\Feature\AuthenticationTest ✓ login with wrong credentials Tests: 1 passed (3 assertions) Duration: 0.75s
TC-006	REQ-001 / REQ-NF-005	Nome: Accesso a rotta protetta senza autenticazione (middleware auth) Risultato atteso: Da browser non autenticato, l'accesso diretto a /projects/1 viene bloccato dal middleware auth e reindirizzato al login senza mostrare il contenuto protetto.	SUPERATO – L'accesso diretto a /projects/1 è stato bloccato dal middleware auth e reindirizzato al login; contenuto protetto non mostrato. Esito conforme al risultato atteso. PASS Tests\Feature\AuthenticationTest ✓ safe route without authentication Tests: 1 passed (2 assertions) Duration: 0.49s
TC-007	REQ-002	Nome: L'Admin accede alla pagina di amministrazione utenti Risultato atteso: La pagina /admin/users è accessibile al solo ruolo Admin e mostra l'elenco di tutti gli account con le relative azioni di gestione (CRUD).	SUPERATO – La pagina /admin/users è stata visualizzata dall'Admin con l'elenco completo degli account e le azioni CRUD. Esito conforme al risultato atteso. PASS Tests\Feature\AdminUsersTest ✓ admin can access users page Tests: 1 passed (2 assertions) Duration: 0.58s
TC-008	REQ-002 / REQ-NF-005	Nome: L'utente standard non può accedere all'amministrazione utenti Risultato atteso: Un User che tenta di accedere a /admin/users riceve un errore HTTP 403 (Forbidden); la pagina di amministrazione non viene mostrata.	SUPERATO – L'accesso dell'User a /admin/users è stato negato con errore HTTP 403; la pagina non è stata mostrata. Esito conforme al risultato atteso. PASS Tests\Feature\AdminUsersTest ✓ standard user cannot access admin page Tests: 1 passed (1 assertions) Duration: 0.59s

Test Case	Riferimento	Descrizione	Risultato effettivo
TC-009	REQ-002	Nome: L'Admin modifica il ruolo di sistema di un utente Risultato atteso: L'Admin cambia il ruolo di un utente (es. da User ad Admin); nel DB il campo role risulta aggiornato e l'utente acquisisce i relativi permessi.	SUPERATO – Il ruolo è stato aggiornato: nel DB il campo role risulta “admin” e l'utente ha acquisito i relativi permessi. Esito conforme al risultato atteso. PASS Tests\Feature\AdminUsersTest ✓ admin can update user role Tests: 1 passed (2 assertions) Duration: 0.56s
TC-010	REQ-003	Nome: Creazione di un progetto con dati validi Risultato atteso: Il progetto viene creato con stato “attivo” e compare nella lista; il creatore risulta membro con ruolo Project Manager e viene creata la board con le colonne di default.	SUPERATO – Il progetto è stato creato (stato “attivo”) e compare in lista; il creatore risulta Project Manager ed è stata generata la board con le colonne di default. Esito conforme al risultato atteso. PASS Tests\Feature\ProjectCRUDAndPermissionsTest ✓ project creation with valid data Tests: 1 passed (6 assertions) Duration: 0.56s
TC-011	REQ-003 / REQ-NF-001	Nome: Creazione progetto con titolo duplicato Risultato atteso: La creazione con titolo già esistente viene rifiutata con errore sul campo titolo; i dati inseriti rimangono nel form e nessun progetto duplicato viene creato.	SUPERATO – La creazione è stata rifiutata con errore sul campo titolo; i dati del form sono stati mantenuti e nessun progetto duplicato è stato creato. Esito conforme al risultato atteso. PASS Tests\Feature\ProjectCRUDAndPermissionsTest ✓ project creation with duplicate name Tests: 1 passed (3 assertions) Duration: 0.56s
TC-012	REQ-003	Nome: Archiviazione progetto con conferma (soft delete) Risultato atteso: Prima dell'archiviazione viene mostrato un modal di conferma con avviso di irreversibilità; dopo la conferma lo stato diventa “archiviato” e il progetto risulta di sola lettura.	SUPERATO – È stato mostrato il modal di conferma con avviso; dopo la conferma lo stato nel DB è diventato “archiviato” e il progetto è di sola lettura. Esito conforme al risultato atteso. PASS Tests\Feature\ProjectCRUDAndPermissionsTest ✓ project archiving Tests: 1 passed (3 assertions) Duration: 0.53s

Test Case	Riferimento	Descrizione	Risultato effettivo
TC-013	REQ-003	Nome: Modifica di un progetto archiviato non consentita Risultato atteso: Un progetto archiviato resta visualizzabile in sola lettura; qualsiasi tentativo di modifica (anche forzando la rotta) viene bloccato lato server e i dati non vengono alterati.	SUPERATO – Il progetto archiviato è rimasto in sola lettura; il tentativo di modifica forzata via rotta è stato respinto lato server e i dati non sono stati alterati. Esito conforme al risultato atteso. PASS Tests\Feature\ProjectCRUDAndPermissionsTest ✓ archived project update is blocked Tests: 1 passed (2 assertions) Duration: 0.55s
TC-014	REQ-003 / REQ-NF-005	Nome: Un Viewer non può eliminare o modificare un progetto Risultato atteso: I pulsanti di modifica ed eliminazione non sono visibili al Viewer; l'accesso diretto alla rotta restituisce HTTP 403 e il progetto non viene modificato né eliminato.	SUPERATO – I pulsanti di modifica/eliminazione non erano visibili al Viewer; l'accesso diretto alla rotta ha restituito HTTP 403 e il progetto non è stato alterato. Esito conforme al risultato atteso. PASS Tests\Feature\ProjectCRUDAndPermissionsTest ✓ viewer cannot delete project Tests: 1 passed (2 assertions) Duration: 0.59s
TC-015	REQ-004	Nome: Il Project Manager invita un nuovo membro assegnando un ruolo Risultato atteso: Il PM invita un utente registrato assegnandogli un ruolo (es. Developer); l'utente viene aggiunto come membro del progetto e compare nella lista con il ruolo corretto.	SUPERATO – L'utente è stato aggiunto come membro con ruolo Developer e compare nella lista membri con il ruolo corretto. Esito conforme al risultato atteso. PASS Tests\Feature\MemberManagementAndRolePermissionsTest ✓ pm invites new member Tests: 1 passed (2 assertions) Duration: 0.65s
TC-016	REQ-004	Nome: Il Project Manager modifica il ruolo di un membro e lo rimuove Risultato atteso: Il ruolo del membro viene aggiornato (es. da Developer a Viewer); dopo la rimozione il membro non compare più nella lista e perde l'accesso al progetto.	SUPERATO – Il ruolo è stato aggiornato a Viewer; dopo la rimozione il membro non compare più nella lista e ha perso l'accesso al progetto. Esito conforme al risultato atteso. PASS Tests\Feature\MemberManagementAndRolePermissionsTest ✓ pm updates role and removes member Tests: 1 passed (4 assertions) Duration: 0.57s
TC-017	REQ-005	Nome: Il Developer può operare sui task ma non gestire i membri Risultato atteso: Il Developer può creare, modificare e spostare i task e aggiungere commenti/etichette; le funzioni di gestione membri e colonne non sono accessibili.	SUPERATO – Il Developer ha potuto creare, modificare e spostare task e aggiungere commenti/etichette; le funzioni di gestione membri e colonne non erano accessibili. Esito conforme al risultato atteso. PASS Tests\Feature\MemberManagementAndRolePermissionsTest ✓ developer cannot manage columns ✓ developer cannot manage members Tests: 2 passed (2 assertions) Duration: 0.61s

Test Case	Riferimento	Descrizione	Risultato effettivo
TC-018	REQ-005	Nome: Il Viewer ha accesso in sola lettura Risultato atteso: Il Viewer visualizza progetto, board, task, commenti, activity log e dashboard ma non vede alcuna azione di scrittura; l'accesso diretto a una rotta di scrittura restituisce HTTP 403.	SUPERATO – Il Viewer ha visualizzato tutti i contenuti senza alcuna azione di scrittura; l'accesso diretto a una rotta di scrittura ha restituito HTTP 403. Esito conforme al risultato atteso. PASS Tests\Feature\MemberManagementAndRolePermissionsTest ✓ viewer task write returns403 Tests: 1 passed (1 assertions) Duration: 0.54s
TC-019	REQ-006	Nome: Selezione tipologie colonne di default Risultato atteso: Alla creazione di una colonna il modale mostra le tipologie di default ("Backlog", "Da fare", "In corso", "In revisione", "Testing", "Completato") nell'ordine corretto.	SUPERATO – Il modale ha mostrato le tipologie di default nell'ordine corretto (Backlog, Da fare, In corso, In revisione, Testing, Completato). Esito conforme al risultato atteso. PASS Tests\Feature\KanbanColumnsTest ✓ default column types available for selection Tests: 1 passed (9 assertions) Duration: 0.56s
TC-020	REQ-006	Nome: Il Project Manager aggiunge, riordina ed elimina una colonna Risultato atteso: La nuova colonna viene creata, riordinata nella posizione richiesta ed eliminata correttamente; la board riflette le modifiche in modo persistente.	SUPERATO – La colonna è stata creata, riordinata ed eliminata correttamente; la board ha riflesso le modifiche in modo persistente. Esito conforme al risultato atteso. PASS Tests\Feature\KanbanColumnsTest ✓ pm adds reorders and deletes column Tests: 1 passed (7 assertions) Duration: 0.58s
TC-021	REQ-006	Nome: Eliminazione di una colonna contenente task Risultato atteso: Il sistema richiede conferma; dopo la conferma la colonna viene eliminata insieme a tutti i task associati, senza lasciare dati inconsistenti.	SUPERATO – È stata richiesta conferma; dopo la conferma la colonna e tutti i task associati sono stati eliminati senza dati inconsistenti. Esito conforme al risultato atteso. PASS Tests\Feature\KanbanColumnsTest ✓ delete column with tasks also deletes tasks Tests: 1 passed (4 assertions) Duration: 0.57s
TC-022	REQ-007	Nome: Creazione task con assegnatario di default Risultato atteso: Il task viene creato nella colonna scelta; non avendo selezionato un assegnatario, il campo assignee_id corrisponde all'utente creatore.	SUPERATO – Il task è stato creato nella colonna "Da fare"; non avendo scelto un assegnatario, assignee_id corrisponde al creatore. Esito conforme al risultato atteso. PASS Tests\Feature\TaskCRUDAndMoveTest ✓ task creation with creator as default assignee Tests: 1 passed (4 assertions) Duration: 0.72s

Test Case	Riferimento	Descrizione	Risultato effettivo
TC-023	REQ-007	Nome: Creazione task con assegnatario non membro del progetto Risultato atteso: Il tentativo di assegnare il task a un utente non membro del progetto viene rifiutato con errore sul campo assegnatario; il task non viene salvato.	SUPERATO – Il tentativo di assegnare a un utente non membro è stato rifiutato con errore sul campo assegnatario; il task non è stato salvato. Esito conforme al risultato atteso. PASS Tests\Feature\TaskCRUDAndMoveTest ✓ task creation with non member assignee is rejected Tests: 1 passed (2 assertions) Duration: 0.58s
TC-024	REQ-007	Nome: Modifica ed eliminazione di un task con conferma Risultato atteso: Le modifiche ai campi del task vengono salvate e visualizzate; l'eliminazione richiede conferma e, dopo conferma, il task non compare più nella board né nel DB.	SUPERATO – Le modifiche sono state salvate e visualizzate; l'eliminazione ha richiesto conferma e, dopo conferma, il task è sparito da board e DB. Esito conforme al risultato atteso. PASS Tests\Feature\TaskCRUDAndMoveTest ✓ task update and deletion Tests: 1 passed (4 assertions) Duration: 0.58s
TC-025	REQ-008	Nome: Spostamento di un task tra colonne dall'interfaccia Risultato atteso: Spostando un task tra colonne (drag&drop o select), il task appare nella nuova colonna e nel DB il campo column_id viene aggiornato con ordine coerente.	SUPERATO – Lo spostamento del task ha aggiornato la colonna in interfaccia e il campo column_id nel DB, con ordine coerente. Esito conforme al risultato atteso. PASS Tests\Feature\TaskCRUDAndMoveTest ✓ task move between columns Tests: 1 passed (2 assertions) Duration: 0.67s
TC-026	REQ-009	Nome: Presenza delle etichette di default e loro non modificabilità Risultato atteso: Sono presenti le etichette di default (DIAGRAMMA, ANALISI, BACKEND, FRONTEND, DOCUMENTAZIONE, TEST); le azioni di modifica/eliminazione non sono disponibili e il tentativo forzato viene respinto.	SUPERATO – Le etichette di default erano presenti; le azioni di modifica/eliminazione non erano disponibili e il tentativo forzato è stato respinto. Esito conforme al risultato atteso. PASS Tests\Feature\LabelsTest ✓ system labels are present and protected Tests: 1 passed (10 assertions) Duration: 0.60s

Test Case	Riferimento	Descrizione	Risultato effettivo
TC-027	REQ-009 / REQ-NF-001	Nome: Creazione etichetta personalizzata con validazione del colore Risultato atteso: Un'etichetta con colore esadecimale valido (es. #FF0000) viene creata e associabile ai task; un colore non valido viene rifiutato con messaggio di errore sul formato.	SUPERATO – Caso A: l'etichetta "Urgente" è stata creata e associata a un task. Caso B: il colore non valido è stato rifiutato con errore sul formato. Esito conforme al risultato atteso. PASS Tests\Feature\LabelsTest ✓ custom label creation with color validation Tests: 1 passed (6 assertions) Duration: 0.59s
TC-028	REQ-010	Nome: Gestione checklist di sotto-attività di un task Risultato atteso: Le sotto-attività con titolo vengono aggiunte; quella senza titolo viene rifiutata (titolo obbligatorio). Dopo il completamento di una, il task mostra il conteggio "1 / 2".	SUPERATO – Le sotto-attività con titolo sono state aggiunte; quella senza titolo è stata rifiutata e il task ha mostrato il conteggio "1 / 2". Esito conforme al risultato atteso. PASS Tests\Feature\SubtaskTest ✓ subtask add complete and count Tests: 1 passed (5 assertions) Duration: 0.65s
TC-029	REQ-011 / REQ-013	Nome: Commento con menzione @username genera notifica Risultato atteso: Una menzione (@luca.verdi) nel commento/descrizione viene riconosciuta; per l'utente menzionato viene generata una notifica via email relativa alla menzione.	FALLITO – La menzione non è stata riconosciuta nella descrizione, la struttura del database non permette il salvataggio delle menzioni nell'apposita tabella. La feature dei commenti e delle menzioni non è stata ancora implementata. FAIL Tests\Feature\ActivityLogTest ✗ task description mention is persisted
TC-030	REQ-012 / criterio 224	Nome: Registrazione degli eventi rilevanti nell'activity log Risultato atteso: Le azioni rilevanti (creazione task, assegnazione, cambio di stato) vengono registrate nel log; ogni voce contiene utente autore, tipo evento, task, timestamp e dati di contesto.	SUPERATO – Nell'activity log sono comparse le voci di creazione, assegnazione e cambio di stato, ciascuna con autore, tipo evento, task, timestamp e contesto. Esito conforme al risultato atteso. PASS Tests\Feature\ActivityLogTest ✓ activity log records task events Tests: 1 passed (4 assertions) Duration: 0.79s

Test Case	Riferimento	Descrizione	Risultato effettivo
TC-031	REQ-012	Nome: Activity log filtrabile e accessibile a tutti i membri Risultato atteso: Il Viewer può visualizzare l'activity log del progetto in sola lettura; le voci mostrate appartengono esclusivamente al progetto selezionato.	SUPERATO – Il Viewer ha visualizzato l'activity log in sola lettura, con voci appartenenti esclusivamente al progetto selezionato. Esito conforme al risultato atteso. PASS Tests\Feature\ActivityLogTest ✓ activity log is accessible to viewer Tests: 1 passed (4 assertions) Duration: 0.59s
TC-032	REQ-013	Nome: Notifica email all'assegnazione di un task Risultato atteso: Assegnando un task a un membro, viene generata e inviata un'email di notifica all'assegnatario contenente il riferimento al task assegnato.	SUPERATO – All'assegnazione è stata generata e inviata l'email di notifica all'assegnatario, con il riferimento al task. Esito conforme al risultato atteso. PASS Tests\Feature\EmailNotificationTest ✓ task assignment sends email notification Tests: 1 passed (1 assertions) Duration: 0.61s
TC-033	REQ-013	Nome: Notifica email all'avvicinarsi della scadenza di un task Risultato atteso: Eseguendo il comando schedulato di controllo scadenze, viene inviata un'email di notifica all'assegnatario per la scadenza imminente; i task non interessati non generano invii errati.	SUPERATO – Eseguendo lo scheduler, è stata inviata l'email di scadenza imminente all'assegnatario; i task non interessati non hanno generato invii. Esito conforme al risultato atteso. PASS Tests\Feature\EmailNotificationTest ✓ due soon notification sent by scheduler Tests: 1 passed (1 assertions) Duration: 0.55s
TC-034	REQ-013	Nome: Notifica email al cambio di stato di un task assegnato Risultato atteso: Spostando un task assegnato in un'altra colonna/stato, viene inviata un'email di notifica all'assegnatario relativa al cambiamento di stato.	SUPERATO – Lo spostamento del task assegnato ha generato l'email di notifica all'assegnatario relativa al cambio di stato. Esito conforme al risultato atteso. PASS Tests\Feature\EmailNotificationTest ✓ task move sends email notification to assignee Tests: 1 passed (1 assertions) Duration: 0.60s

Test Case	Riferimento	Descrizione	Risultato effettivo
TC-035	REQ-014	Nome: Dashboard di progetto con statistiche corrette Risultato atteso: La dashboard mostra valori coerenti con i dati noti: percentuale di completamento, conteggi per stato, elenco task scaduti e numero di task per membro corretti.	SUPERATO – La dashboard ha mostrato valori coerenti con i dati noti: percentuale di completamento, conteggi per stato, task scaduti e task per membro corretti. Esito conforme al risultato atteso. PASS Tests\Feature\DashboardTest ✓ dashboard shows correct statistics Tests: 1 passed (7 assertions) Duration: 0.63s
TC-036	REQ-014 / REQ-NF-004	Nome: Dashboard con progetto senza task (caso limite) Risultato atteso: Con un progetto senza task la dashboard viene visualizzata senza errori: percentuale 0% (senza divisione per zero), conteggi a 0 ed elenco task scaduti vuoto.	SUPERATO – La dashboard del progetto vuoto è stata visualizzata senza errori: 0% di completamento, conteggi a 0 ed elenco scaduti vuoto. Esito conforme al risultato atteso. PASS Tests\Feature\DashboardTest ✓ dashboard with empty project shows zero without error Tests: 1 passed (2 assertions) Duration: 0.57s
TC-037	REQ-015	Nome: Esportazione del report PDF della dashboard Risultato atteso: Viene generato e scaricato un PDF con i metadati del progetto, le statistiche della dashboard e la lista dei task con il relativo stato; il documento è leggibile e completo.	FALLITO – La feature di generazione del report PDF non è stata implementata.
TC-038	REQ-NF-001 / criterio 194	Nome: Validazione form con ripopolamento dei dati e nessun salvataggio parziale Risultato atteso: Inviando il form con il titolo obbligatorio vuoto, il form viene rifiutato lato server: campo evidenziato con messaggio, valori inseriti ripopolati e nessun progetto parziale salvato.	SUPERATO – Il form è stato rifiutato lato server: campo titolo evidenziato, descrizione e priorità ripopolate e nessun progetto parziale salvato nel DB. Esito conforme al risultato atteso. PASS Tests\Feature\ValidationAndSecurityTest ✓ form validation error shows old input and no partial save Tests: 1 passed (5 assertions) Duration: 0.53s

Test Case	Riferimento	Descrizione	Risultato effettivo
TC-039	REQ-NF-002 / criterio 164	Nome: Pagine di errore personalizzate 403 e 404 Risultato atteso: Le rotte inesistenti o non autorizzate mostrano pagine di errore personalizzate (404 e 403) con messaggi comprensibili, senza esporre stack trace o dettagli tecnici interni.	SUPERATO – Sono state mostrate pagine di errore personalizzate 404 e 403 con messaggi comprensibili, senza stack trace né dettagli tecnici. Esito conforme al risultato atteso. PASS Tests\Feature\ValidationAndSecurityTest ✓ custom404 page returned ✓ custom403 page returned for unauthorized route Tests: 2 passed (2 assertions) Duration: 0.60s
TC-040	REQ-NF-003 / criterio 146	Nome: Feedback visivo e conferma per azioni distruttive Risultato atteso: Dopo le azioni viene mostrato un feedback con colorazione semantica (verde successo / rosso errore); le azioni distruttive richiedono conferma esplicita via modal e l'annullamento non le esegue.	SUPERATO – Dopo le azioni è stato mostrato il feedback con colorazione semantica; le azioni distruttive hanno richiesto conferma via modal e l'annullamento non le ha eseguite. Esito conforme al risultato atteso. PASS Tests\Feature\ValidationAndSecurityTest ✓ success flash message after project creation Tests: 1 passed (1 assertions) Duration: 0.54s
TC-041	REQ-NF-005 / criterio 164	Nome: Protezione CSRF sui form Risultato atteso: Una richiesta POST priva di token CSRF valido viene respinta dal server (errore HTTP 419 / Page Expired) e l'operazione non viene eseguita.	SUPERATO – La richiesta POST senza token CSRF valido è stata respinta con errore HTTP 419 (Page Expired) e l'operazione non è stata eseguita. Esito conforme al risultato atteso. PASS Tests\Feature\ValidationAndSecurityTest ✓ csrf middleware is registered for web routes Tests: 1 passed (2 assertions) Duration: 0.46s
TC-042	REQ-NF-005 / criterio 237	Nome: Protezione contro XSS nell'output Risultato atteso: Un titolo contenente <code><script>alert('XSS')</script></code> viene mostrato come testo letterale (escape automatico Blade <code>{{ }}</code>); lo script non viene eseguito e non compare alcun popup.	SUPERATO – Il titolo con <code><script>alert('XSS')</script></code> è stato mostrato come testo letterale tramite escape Blade; lo script non è stato eseguito e nessun popup è comparso. Esito conforme al risultato atteso. PASS Tests\Feature\ValidationAndSecurityTest ✓ xss payload in task title is stored literally Tests: 1 passed (4 assertions) Duration: 0.73s

5.3 Mancanze/limitazioni conosciute

Di seguito vengono esposte le limitazioni note e consapevoli, derivanti dalle priorità assegnate ai requisiti in fase di analisi e dal tempo a disposizione. Si precisa che tutti i requisiti funzionali di priorità 1 risultano completamente implementati, le mancanze riguardano esclusivamente requisiti di priorità 2 e 3 e alcuni aspetti non funzionali secondari.

Commenti e menzioni: Non è stato implementato il sistema di commenti sui task con menzione @utente e relativa notifica. La collaborazione testuale è quindi affidata al solo campo descrizione del task, privo di riconoscimento delle menzioni e di generazione automatica della notifica all'utente menzionato.

Esportazione del report in PDF: Non è stata implementata la generazione e il download del report PDF di riepilogo della dashboard di progetto. Le statistiche (percentuale di completamento, task per stato, task scaduti, task per membro) restano comunque consultabili direttamente nella dashboard dell'applicazione.

Invio delle notifiche email in modalità sincrona: Le notifiche legate alle azioni dirette dell'utente (assegnazione, spostamento, invito, creazione utente) vengono inviate in modo sincrono all'interno della richiesta HTTP, anziché tramite una coda asincrona. Questa scelta, descritta e motivata nel capitolo di implementazione rende il comportamento prevedibile in sviluppo e in fase di test e non richiede un worker di coda sempre attivo. In un contesto di produzione ad alto volume di traffico costituisce tuttavia un limite, poiché un rallentamento del mail server si ripercuote sul tempo di risposta percepito dall'utente. L'architettura predispone già il passaggio a una coda persistente (driver database o Redis) agendo sulla sola configurazione, senza modifiche al codice applicativo.

Esperienza su dispositivi mobili: Il layout è stato progettato e ottimizzato per desktop e tablet. Su smartphone l'applicazione resta utilizzabile, ma la board kanban richiede uno scorrimento orizzontale per consultare tutte le colonne, come dichiarato nei requisiti non funzionali. Non è stata realizzata una vista mobile dedicata (ad esempio a colonne impilate verticalmente), in quanto il contesto d'uso previsto per lo strumento è prevalentemente desktop.

Assenza di una landing page pubblica: L'applicazione non dispone di una pagina di presentazione pubblica: l'accesso alla radice del sito reindirizza direttamente alla pagina di login per gli utenti non autenticati o alla panoramica progetti per quelli già autenticati. Trattandosi di uno strumento gestionale interno destinato a utenti registrati, una landing page promozionale non rientrava negli obiettivi del progetto.

6 Consuntivo

La pianificazione iniziale suddivideva le 80 ore in quattro fasi: Analisi e Progettazione, Implementazione, Test e Documentazione. Il consuntivo conferma le ore impiegate pianificate nel preventivo per queste categorie.

Le differenze principali riguardano la sequenza delle attività: durante l'implementazione le sotto-attività sono state in parte sovrapposte e riordinate in base alle esigenze emerse durante lo sviluppo, anziché svolte in blocco come previsto. L'autenticazione, ad esempio, è stata completata subito grazie a Laravel Breeze (con un notevole risparmio di tempo), mentre la parte di autorizzazione tramite middleware è stata sviluppata più avanti.

Alcune attività si sono dimostrate più lunghe del previsto, in particolare lo scheduler delle notifiche, che ha causato circa due ore di ritardo per problemi di concorrenza tra processi, risolti passando dall'invio email asincrono a quello sincrono. Anche lo schema del database ha richiesto revisioni non pianificate (soft delete sulla tabella ponte progetti-utenti, campo position sui task), questo perché durante la fase d'implementazione ci si accorge di elementi e problemi che in fase di progettazione non emergono.

La fase di Test, prevista interamente il 22.05, è in parte slittata al 26.05 a causa dei ritardi accumulati, ma è stata completamente recuperata in giornata. La Documentazione, pur concentrata nelle ultime due giornate, è stata avviata in parallelo fin dall'inizio attraverso l'abbozzo di diagrammi e tabelle dei requisiti.

Nel complesso il bilancio rispetta le 80 ore preventive: il tempo risparmiato in alcune attività ha compensato i ritardi in altre. In linea di massima possiamo osservare che il diagramma consuntivo rispetta la maggior parte delle tempistiche.

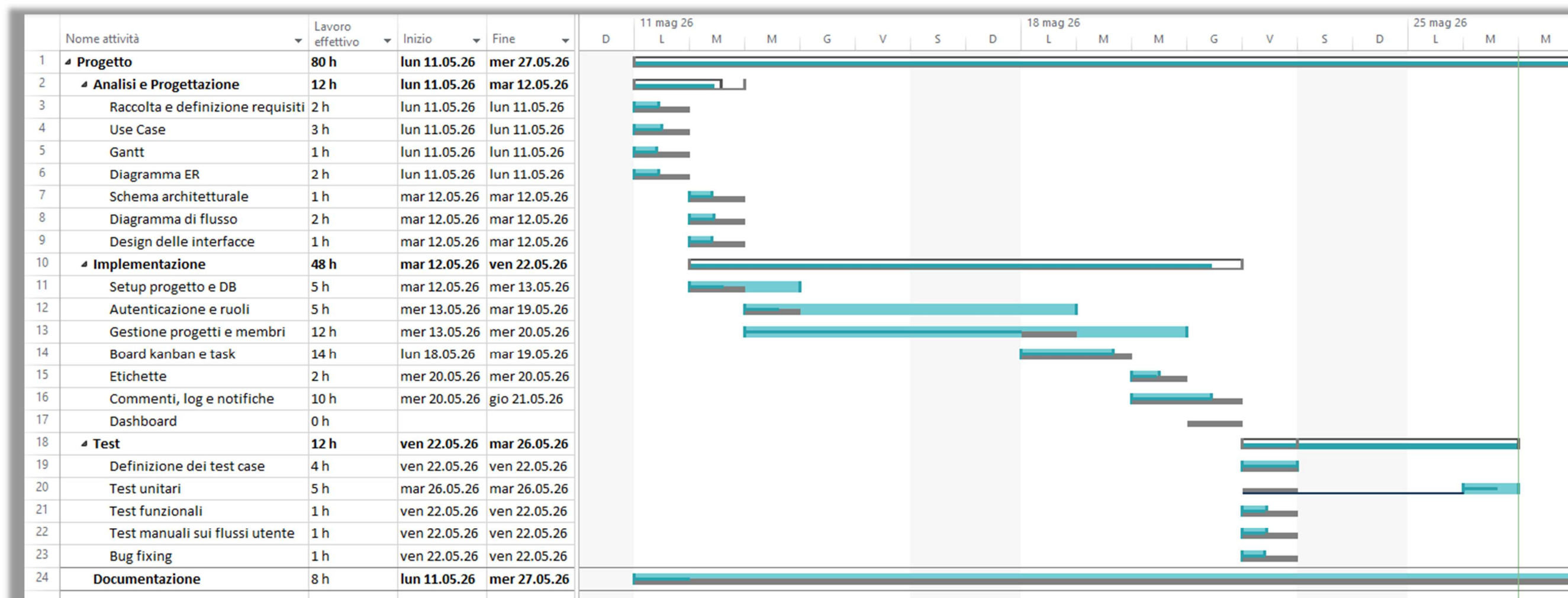


Figura 38 - Gantt consuntivo

7 Conclusioni

BrainFlow raggiunge l'obiettivo prefissato: tutti i requisiti di priorità 1 sono stati implementati e verificati tramite i test, con validazione dei dati, gestione degli errori e interfaccia intuitiva trattati come requisiti centrali. BrainFlow non introduce funzionalità nuove rispetto a prodotti commerciali come Trello o Jira, ma le ripropone con uno strumento autonomo e controllabile, adatto a team piccoli o contesti scolastici per cui le piattaforme commerciali risultino avere troppe funzionalità o problematiche per l'archiviazione dei dati su terze parti. Il risultato è un prodotto completo e funzionante pensato con l'ideologia di un uso scolastico, ma adattabile anche a contesti lavorativi.

7.1 Sviluppi futuri

Di seguito vengono raccolti i possibili miglioramenti del prodotto, individuati durante lo sviluppo e organizzati per area. Si trattano di migliorie di funzionalità già implementate e che renderebbero l'applicazione più robusta, performante e adatta a un contesto di produzione ad alto volume.

Gestione degli errori e integrità dei dati: Le operazioni che coinvolgono più scritture sul database come la creazione di un task con assegnatari e sotto-attività potrebbero essere incapsulate in transazioni, così da garantire che un fallimento non lasci dati parziali. Sarebbe inoltre opportuno uniformare il formato delle risposte di errore restituite dai controller, semplificando la gestione lato frontend.

Aggiornamenti in tempo reale: Attualmente la board aggiorna i cambiamenti solo al ricaricamento della pagina. Se più utenti operano sullo stesso progetto, ciascuno vede lo stato precedente finché non aggiorna manualmente. Uno sviluppo futuro è propagare in tempo reale gli eventi rilevanti (spostamento di un task, cambio di stato, creazione di una scheda) sulle board di tutti gli utenti collegati, tramite il sistema di broadcasting di Laravel abbinato a un WebSocket (ad esempio Laravel Reverb o Pusher). Ciò eviterebbe disallineamenti tra le viste e ridurrebbe i conflitti dovuti a modifiche concorrenti.

Arricchimento dell'activity log: L'activity log attuale registra autore, tipo di azione e istante. Un'estensione utile è aumentare il dettaglio memorizzato, salvando per esempio i campi aggiornati rispetto alla versione precedente. Questo permetterebbe di ricostruire con precisione la storia di ogni task e di rendere il log consultabile, eventualmente affiancato da filtri per tipo di evento, membro e periodo.

Validazione: Alcuni controlli possono essere irrobustiti come per esempio l'introduzione di un limite massimo al numero di sotto-attività per task, per evitare input anomali.

Sicurezza: Si prevede l'introduzione di un meccanismo di rate limiting (limite di richieste) sugli endpoint sensibili (login, registrazione, reset password e operazioni di amministrazione) come protezione dagli attacchi di forza bruta. Inoltre la procedura di generazione delle password amministrative potrebbe essere rivista per evitare la presenza della password in chiaro nei log di posta.

Performance: Su progetti di grandi dimensioni converrebbe introdurre la paginazione nel caricamento dei task e dell'activity log, oggi caricati per intero in memoria, e aggiungere indici espliciti sulle colonne più usate nei filtri (ad esempio project_id).

Esperienza utente: L'interazione drag-and-drop potrebbe beneficiare di aggiornamenti ottimistici dell'interfaccia, di un feedback visivo esplicito in caso di errore di rete e di un meccanismo per gestire correttamente lo spostamento rapido di più task in sequenza.

7.2 Considerazioni personali

Alla conclusione dello sviluppo di BrainFlow posso dire che sono soddisfatto pienamente del risultato ottenuto. Durante lo sviluppo non ho appreso grandi competenze tecniche, siccome ho già trattato progetti di questo tipo in Laravel, ma comunque reputo che l'insegnamento più grande è stato la gestione del progetto e del tempo. Infatti, nei progetti passati, ho quasi sempre fatto fatica a gestire il tempo, ma in questo, sono riuscito a fare una pianificazione in linea. Sono riuscito a concentrarmi di più sullo svolgere al meglio attività primarie piuttosto di iniziare attività secondarie e non completarle. Ho imparato inoltre a capire quando il risultato di una funzionalità sia "abbastanza buono". Non ho incontrato difficoltà particolari, l'unica che mi ha preso tanto tempo è stata la configurazione del worker per la gestione dell'invio mail tramite chiamate asincrone, che però alla fine, ho deciso di rimuovere. Trovo che questo progetto mi abbia permesso di sviluppare una crescita personale sul metodo di lavoro e che avere un prodotto funzionante con le funzionalità necessarie è meglio di un prodotto con bug ma con molte funzionalità implementate.

8 Glossario

Termine	Descrizione
Backlog	in ambito kanban, colonna o elenco che raccoglie i task ancora da pianificare o non ancora presi in carico, in attesa di essere lavorati.
bcrypt	algoritmo di hashing delle password progettato per essere lento e resistente agli attacchi a forza bruta; in Laravel produce un hash riconoscibile dal prefisso \$2y\$.
Blade	template engine (motore di modelli) di Laravel che permette di scrivere le pagine HTML inserendo logica di presentazione e dati dinamici; la sintassi {{ }} effettua automaticamente l'escape dell'output.
Board kanban	tabella visuale suddivisa in colonne che rappresentano gli stati del flusso di lavoro, sulla quale i task vengono rappresentati come schede spostabili da una colonna all'altra.
cascadeOnDelete	vincolo sulle foreign key che, all'eliminazione di una riga "padre", elimina automaticamente anche le righe "figlie" collegate (es. i task di un progetto eliminato).
CRUD	Create, Read, Update, Delete: insieme delle quattro operazioni fondamentali sulla gestione dei dati: creazione, lettura, aggiornamento ed eliminazione.
CSRF	Cross-Site Request Forgery: attacco in cui un sito malevolo induce il browser dell'utente a inviare richieste non autorizzate a un'applicazione su cui è autenticato; Laravel lo previene tramite un token di sicurezza nei form.
CSS	Cascading Style Sheets: linguaggio che permette di definire il layout, i colori e la grafica di una pagina web.
Diagramma ER	Entity-Relationship: diagramma entità-relazione che rappresenta graficamente le tabelle del database, i loro attributi e le relazioni che le legano tra loro.
Drag & drop	interazione dell'interfaccia grafica che consente di spostare un elemento (es. una scheda task) trascinandolo con il mouse e rilasciandolo nella posizione desiderata.
Eloquent ORM	Object-Relational Mapping: il sistema di mappatura oggetto-relazionale di Laravel, che rappresenta le tabelle del database come classi (modelli) e le righe come oggetti, evitando di scrivere query SQL manuali.
.env	file di configurazione di Laravel che contiene le variabili d'ambiente dell'applicazione (es. credenziali del database e del server SMTP), tenute separate dal codice sorgente.
Esadecimale (hex)	sistema di numerazione in base 16 usato per rappresentare i colori in ambito web nel formato #RRGGBB (es. #FF0000 per il rosso).
Flash message	messaggio temporaneo mostrato all'utente una sola volta dopo un'azione (es. "progetto creato"), memorizzato in sessione e poi rimosso automaticamente.

Termine	Descrizione
FlowChart	diagramma di flusso che rappresenta graficamente i passaggi e le decisioni di un processo tramite simboli standardizzati collegati da frecce.
Foreign key	chiave esterna; campo di una tabella che fa riferimento alla chiave primaria di un'altra tabella, stabilendo una relazione e garantendo l'integrità dei dati.
Gantt	diagramma a barre orizzontali usato nella pianificazione di progetto per rappresentare le attività e la loro distribuzione nel tempo.
Hash	valore di lunghezza fissa generato da una funzione matematica a senso unico a partire da un dato (es. una password); non è reversibile, quindi non permette di risalire al valore originale.
HTTP / HTTPS	HyperText Transfer Protocol (Secure) : protocollo di comunicazione usato dai browser e dai server web per scambiarsi le pagine; nella variante HTTPS la comunicazione è cifrata e quindi sicura.
HTTP 403	Forbidden : codice di stato HTTP che indica che l'utente è autenticato ma non ha i permessi necessari per accedere alla risorsa richiesta.
HTTP 404	Not Found : codice di stato HTTP che indica che la risorsa o la pagina richiesta non esiste sul server.
HTTP 419	Page Expired : codice di stato usato da Laravel quando una richiesta viene respinta perché priva di un token CSRF valido o con sessione scaduta.
HTTP 500	Internal Server Error : codice di stato HTTP che indica un errore generico interno al server durante l'elaborazione della richiesta.
Job / Queue	meccanismo di Laravel per eseguire operazioni in modo asincrono tramite code: un job (compito) viene messo in coda ed elaborato in background, utile ad esempio per l'invio delle email senza rallentare la risposta all'utente.
Kanban	metodologia di gestione del lavoro di origine giapponese che visualizza le attività come schede su una board suddivisa in colonne, per rendere immediato lo stato di avanzamento del flusso.
Laravel	framework di sviluppo web open source basato sul linguaggio PHP, utilizzato per realizzare sia la logica applicativa (backend) sia le pagine (frontend) tramite il template engine Blade.
Mailer	componente di Laravel responsabile della composizione e dell'invio delle email, configurabile con diversi driver (es. SMTP, log).
Mailtrap	servizio che intercetta le email inviate in fase di sviluppo o test, permettendo di verificarne il contenuto senza recapitarle realmente ai destinatari.
Menzione (@username)	riferimento a un utente all'interno di un testo (commento o descrizione) tramite il formato @nomeutente; genera una notifica all'utente citato.
Middleware	strato di codice che intercetta le richieste HTTP prima che raggiungano la logica applicativa, usato ad esempio per verificare l'autenticazione o i permessi dell'utente.

Termine	Descrizione
Migration	in Laravel, file che descrive in codice la struttura di una tabella del database, permettendo di crearla, modificarla o ricostruirla in modo versionabile e ripetibile.
Modal	finestra di dialogo che compare in primo piano sopra la pagina, oscurando il resto dell'interfaccia, usata per form o richieste di conferma di un'azione.
MySQL	sistema di gestione di basi di dati relazionali (RDBMS) open source, usato per memorizzare e interrogare i dati dell'applicazione in modo strutturato.
Query Builder	componente di Laravel che permette di costruire query al database in modo programmatico e sicuro, con parametri associati che proteggono dalle SQL injection.
Responsive	caratteristica di un'interfaccia web che adatta automaticamente il proprio layout alle dimensioni dello schermo del dispositivo (desktop, tablet, smartphone).
REST	Representational State Transfer: stile architetturale per la progettazione di servizi web che utilizza i metodi standard del protocollo HTTP per accedere e manipolare le risorse.
restrictOnDelete	vincolo sulle foreign key che impedisce l'eliminazione di una riga "padre" finché esistono righe "figlie" collegate, preservando l'integrità dei dati (es. un utente assegnatario di un task).
Scheduler	componente di Laravel che esegue automaticamente comandi a intervalli di tempo prestabiliti, usato qui per controllare periodicamente le scadenze dei task e inviare le notifiche.
SDK	Software Development Kit: insieme di strumenti, librerie e documentazione forniti per facilitare lo sviluppo di software su una determinata piattaforma o tecnologia.
Seeder	in Laravel, classe che popola il database con dati iniziali predefiniti (es. le etichette di default e l'account Admin) dopo l'esecuzione delle migrazioni.
SMTP	Simple Mail Transfer Protocol: protocollo standard utilizzato per l'invio delle email tra i server di posta; l'applicazione si appoggia a un server SMTP esterno per recapitare le notifiche.
Soft delete	tecnica di "cancellazione logica" in cui un record non viene rimosso dal database ma marcato come non più attivo (qui tramite lo stato "archiviato" del progetto), restando consultabile in sola lettura.
SQL injection	attacco informatico in cui un utente malevolo inserisce codice SQL nei campi di input per manipolare le query ed accedere o alterare illecitamente i dati del database.
Stack trace	elenco dettagliato delle chiamate di codice che hanno portato a un errore; utile in fase di sviluppo ma da non esporre all'utente finale per ragioni di sicurezza.
Tailwind CSS	framework CSS "utility-first" che fornisce classi predefinite da combinare direttamente nell'HTML per definire rapidamente lo stile dell'interfaccia.

Termine	Descrizione
Template engine	motore di modelli che genera pagine HTML combinando un modello statico con dati dinamici; in Laravel questo ruolo è svolto da Blade.
Timestamp	marca temporale che registra la data e l'ora esatte in cui si è verificato un evento (es. la creazione di una voce nell'activity log).
Toast	piccola notifica che compare temporaneamente in un angolo dello schermo per dare un feedback all'utente dopo un'azione, scomparendo poi automaticamente.
Token	stringa generata dal sistema che identifica o autorizza in modo sicuro una richiesta o una sessione (es. il token CSRF dei form o il token di reset password).
Token CSRF	valore segreto e univoco inserito nei form da Laravel e verificato dal server per garantire che la richiesta provenga effettivamente dall'applicazione e non da un sito malevolo.
Use case	caso d'uso; descrizione di un'interazione tra un attore (utente o sistema) e l'applicazione, finalizzata al raggiungimento di un obiettivo specifico.
Waterfall	modello di sviluppo software a cascata, in cui le fasi del progetto (analisi, progettazione, implementazione, test) si susseguono in sequenza lineare.
XSS	Cross-Site Scripting: attacco in cui codice script malevolo viene iniettato in una pagina web e poi eseguito nel browser di altri utenti; viene prevenuto effettuando l'escape dell'output mostrato.

9 Sitografia

<https://laravel.com/docs/12.x>, *Laravel – Documentation*, 12-05-2026.

<https://laravel.com/docs/12.x/eloquent>, *Eloquent: Getting Started – Laravel*, 13-05-2026.

<https://laravel.com/docs/12.x/migrations>, *Database: Migrations – Laravel*, 13-05-2026.

<https://laravel.com/docs/12.x/seeding>, *Database: Seeding – Laravel*, 14-05-2026.

<https://laravel.com/docs/12.x/blade>, *Blade Templates – Laravel*, 14-05-2026.

<https://laravel.com/docs/12.x/validation>, *Validation – Laravel*, 15-05-2026.

<https://laravel.com/docs/12.x/authentication>, *Authentication – Laravel*, 15-05-2026.

<https://laravel.com/docs/12.x/authorization>, *Authorization – Laravel*, 16-05-2026.

<https://laravel.com/docs/12.x/middleware>, *Middleware – Laravel*, 16-05-2026.

<https://laravel.com/docs/12.x/csrf>, *CSRF Protection – Laravel*, 18-05-2026.

<https://laravel.com/docs/12.x/queues>, *Queues – Laravel*, 18-05-2026.

<https://laravel.com/docs/12.x/mail>, *Mail – Laravel*, 19-05-2026.

<https://laravel.com/docs/12.x/notifications>, *Notifications – Laravel*, 19-05-2026.

<https://laravel.com/docs/12.x/scheduling>, *Task Scheduling – Laravel*, 20-05-2026.

<https://laravel.com/docs/12.x/errors>, *Error Handling – Laravel*, 20-05-2026.

<https://laravel.com/docs/12.x/eloquent-relationships>, *Eloquent: Relationships – Laravel*, 21-05-2026.

<https://dev.mysql.com/doc/refman/8.0/en/>, *MySQL 8.0 Reference Manual*, 13-05-2026.

<https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html>, *FOREIGN KEY Constraints – MySQL*, 16-05-2026.

<https://www.php.net/manual/en/>, *PHP Manual*, 12-05-2026.

<https://www.php.net/manual/en/function.password-hash.php>, *password_hash – PHP Manual*, 18-05-2026..

<https://www.trello.com>, *Dashboard Trello – Atlassian*, 11-05-2026.

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>, *HTTP response status codes – MDN Web Docs*, 20-05-2026.

10 Allegati

Allegato 1, cartella 1_QdC:



LPI26_QdC_InCe_Arz
ani.docx

Allegato 2, cartella 5_Applicativo/brainflow



README.md